

AUTOBOOK SERIES

# AI 엔지니어 영어 면접 말하기

내 프로젝트로 만드는 실전 영어 답변

자기소개 · 8개 프로젝트 정답지 · 꼬리질문 · 표현·발음

2026

# 목차

## 0. 들어가며

## 1 말하기 기본기

### 1.1 프레임워크와 표현

- 1.1.1 영어로 프로젝트를 말하는 법 (5-Beat 프레임워크)
- 1.1.2 만능 표현과 숫자·발음 드릴

### 1.2 자기소개

- 1.2.1 자기소개 (Tell me about yourself)

## 2 프로젝트 정답지

### 2.1 플랜티넷 프로젝트

- 2.1.1 ReAct + Reflexion 자기개선 에이전트
- 2.1.2 URL 스미싱 탐지 시스템
- 2.1.3 Milvus Agentic RAG 시스템
- 2.1.4 vLLM 자체 서빙 API
- 2.1.5 LLM 평가 파이프라인

### 2.2 AIO20 & 사이드 프로젝트

- 2.2.1 Visit Busan 여행 챗봇
- 2.2.2 Skylife 영화 추천 시스템
- 2.2.3 MarketScope AI 상권분석 SaaS

## 3 공통 질문 대비

### 3.1 행동·공통 질문

- 3.1.1 강점·약점·실패·갈등·지원동기 (STAR)

# 0. 들어가며

이 책은 AI / ML 엔지니어가 자신의 실제 프로젝트를 영어로 설명하는 면접을 준비하기 위한 말하기 교재입니다. 일반적인 회화책과 다르게, 추천 시스템 연구에서 출발해 벡터 검색, 자체 LLM 서빙, 에이전트, 그리고 운영 LLM 평가 파이프라인까지 이어지는 여덟 개의 실제 프로젝트를 그대로 소재로 삼습니다. 정답(모범답안)은 영어로, 표현의 뜻과 발음, 그리고 어떻게 생각하고 말할지에 대한 설명은 한국어로 적었습니다. 최종 목적은 문장 암기가 아니라 영어로 자기 일을 설명하는 힘을 기르는 것입니다.

영어 면접에서 무너지는 가장 큰 이유는 단어가 부족해서가 아니라, 말의 순서를 즉석에서 만들려다 꼬이기 때문입니다. 그래서 이 책은 먼저 고정된 뼈대를 손에 쥐여 줍니다. 모든 프로젝트 답변은 다섯 박자(Headline, Problem, Decision, Hard Part, Result)로 말하고, 자기소개는 네 박자(Present, Thread, Proof, Fit)로 말합니다. 뼈대가 고정되면 머리는 내용에만 집중할 수 있고, 말이 한 번 끊겨도 다음 박자로 자연스럽게 넘어갈 수 있습니다.

이 책은 세 단계로 짜여 있습니다. Phase 1에서는 말하기의 기본기를 세웁니다. 다섯 박자 프레임워크와 만능 표현, 숫자 발음을 익히고, 면접의 첫 질문인 자기소개를 버전별로 준비합니다. Phase 2에서는 여덟 개 프로젝트마다 정답지를 펼칩니다. 각 정답지는 생각 구조, 길이와 난이도가 다른 세 가지 모범답안, 예상 꼬리질문, 핵심 표현, 발음 주의로 구성됩니다. Phase 3에서는 강점과 약점, 실패와 갈등, 지원 동기 같은 공통 질문을 STAR 구조로 대비합니다.

가장 중요한 사용법은 버전을 사다리처럼 오르는 것입니다. 각 프로젝트의 모범답안은 같은 이야기를 깊이만 다르게 풀어낸 세 버전으로 제공됩니다. 약 30초 분량의 쉬운 버전으로 입을 풀고, 1분에서 2분 분량의 표준 버전을 기본값으로 삼은 뒤, 면접관이 더 파고들면 심화 버전의 기술 디테일을 꺼냅니다. 처음부터 모든 것을 쏟지 말고, 후속 질문에 쓸 카드를 남겨 두는 것이 좋습니다. 표준 버전을 입에 붙이는 것을 1차 목표로 삼으십시오.

이 책의 답변은 이력서에 적힌 사실만으로 만들었습니다. 면접에서도 숫자를 부풀리지 마십시오. 특히 스미싱 탐지의 F1 점수는 실환경(불균형 9 대 1)에서 28.79퍼센트, 균형(1 대 1) 데이터에서 96.10퍼센트로, 서로 다른 분포를 기준으로 한 수치입니다. 후속 질문이 들어오면 이 구분을 정확히 말해야 신뢰가 유지됩니다. 또한 부산관광공사 챗봇이나 영화 추천처럼 정량 지표가 없는 프로젝트는 퍼센트를 지어내지 말고, 상용 운영 중이라는 사실이나 추천 다양성 개선 같은 질적 가치로 답해야 합니다. 모르는 것을 물으면 추측하지 말고, 여기까지는 안다는 점과 어떻게 접근하겠다는 방향을 말하는 편이 훨씬 낫습니다.

다음 단원에서는 모든 답변의 뼈대가 되는 다섯 박자 프레임워크부터, 영어로 프로젝트를 말하는 법을 익혀 보겠습니다.

# 1 말하기 기본기

다섯 박자(H-P-D-H-R) 구조로 임의의 프로젝트를 영어로 설명하고, 네 박자 구조로 자기소개를 길이별로 말할 수 있습니다.

이 Phase는 다음 섹션으로 구성됩니다.

## 1.1 프레임워크와 표현

1.1.1 영어로 프로젝트를 말하는 법 (5-Beat 프레임워크)

1.1.2 만능 표현과 숫자·발음 드릴

## 1.2 자기소개

1.2.1 자기소개 (Tell me about yourself)

## 1.1 프레임워크와 표현

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

**1.1.1 영어로 프로젝트를 말하는 법 (5-Beat 프레임워크)** — 다섯 박자(Headline, Problem, Decision, Hard Part, Result) 구조로 임의의 프로젝트를 30초·90초·심화 세 버전으로 영어로 말할 수 있다.

**1.1.2 만능 표현과 숫자·발음 드릴** — 여덟 개 프로젝트의 한 줄 요약과 핵심 숫자를 영어로 또박또박 말할 수 있다.

### 1.1.1 영어로 프로젝트를 말하는 법 (5-Beat 프레임워크)

이 파일은 모든 프로젝트 정답지의 "뼈대"입니다. 먼저 이 구조를 손에 익히면, 어떤 프로젝트를 묻든 같은 틀로 안정적으로 말할 수 있습니다.  
(정답 문장은 영어, 학습용 설명은 한국어로 표기합니다.)

## A. 왜 "구조"가 먼저인가

영어 면접에서 무너지는 이유는 단어가 부족해서가 아니라 **순서를 즉석에서 만들려다** 말이 꼬이기 때문입니다. 모국어가 아니면 즉흥 구성이 어렵습니다. 그래서 우리는 **고정된 5-비트 뼈대**를 외웁니다. 뼈대가 고정되면, 머리는 "내용"에만 집중하면 되고, 말이 끊겨도 다음 비트로 자연스럽게 넘어갑니다.

핵심 원칙 3가지:

1. **구조 > 유창함**. 더듬어도 구조가 있으면 유능해 보입니다.
2. **숫자로 시작하고 숫자로 끝낸다**. 결과 지표(F1 96%, 60만 건, 비용 100% 제거)가 곧 신뢰입니다.
3. **결정과 이유를 말한다**. "무엇을 했다"보다 "왜 그렇게 정했다(트레이드오프)"가 시니어 신호입니다.

## B. The 5-Beat Project Story (핵심 뼈대)

모든 프로젝트 답변은 이 5개 비트 순서로 말합니다. 각 비트마다 **고정 시작 표현(trigger phrase)**을

외워두면 즉석에서도 막히지 않습니다.

| # | Beat                    | 한국어                  | 이 비트의 시작 표현 (trigger)                                               |
|---|-------------------------|----------------------|---------------------------------------------------------------------|
| 1 | Headline                | 한 줄 정의 + 최고의 결과      | "In short, I built X that [result]."                                |
| 2 | Context & Problem       | 이전 상태와 진짜 문제         | "Before this, ... The real problem was ..."                         |
| 3 | Approach & Key Decision | 한 일 + 자랑할 결정 1개 + 이유 | "So I ... The key decision was to ... I chose A over B because ..." |
| 4 | The Hard Part           | 가장 어려웠던 점 + 해결       | "The tricky part was ... I solved it by ..."                        |
| 5 | Result & Insight        | 정량 결과 + 배운 점         | "As a result, ... What I took away was ..."                         |

[1 Headline] — "뭘 만들었고 결과가 뭐다"  
 |  
 [2 Problem] — "왜 필요했나 (전 상태 + 고통)"  
 |  
 [3 Decision] — "어떻게 했나 + 핵심 결정 + 왜 (트레이드오프)"  
 |  
 [4 Hard Part] — "가장 어려웠던 1가지 + 어떻게 뚫었나"  
 |  
 [5 Result] — "숫자 결과 + 한 줄 교훈"

**암기 팁: "H-P-D-H-R"** (Headline, Problem, Decision, Hard part, Result).  
 면접 중에는 손가락으로 5개를 세며 속으로 체크하세요.

## C. 길이별 버전은 이 뼈대에서 나온다

세 가지 버전은 다른 내용이 아니라, **같은 5-비트를 얼마나 펼치느냐**의 차이입니다.

| 버전                      | 길이     | 구성                           | 언제 쓰나                                      |
|-------------------------|--------|------------------------------|--------------------------------------------|
| ● Version A — Elevator  | 25~35초 | Beat 1 + Beat 5 (헤드라인 + 결과)  | "Tell me about a project." 첫 답, 또는 시간 없을 때 |
| ● Version B — Standard  | 60~90초 | 5비트 각 1~2문장                  | 기본값. 대부분의 "이 프로젝트 설명해줘"에                   |
| ● Version C — Deep Dive | 2~3분   | 5비트 확장 + Beat 3-4에 기술 디테일 1개 | "더 자세히", "어떻게 구현했나" 후속 질문                  |

전략: **항상 Version B로 시작** → 면접관이 더 파면 Version C의 디테일을 꺼냅니다. 먼저 다 쏟지 말고, 후속 질문용 카드를 남겨주세요.

## D. 즉석에서 생각하는 법 (Thinking on your feet)

영어가 막힐 때를 위한 안전장치입니다.

### 1) 시간 버는 표현 (Buying time) — 침묵 대신 이걸 말하세요

- "That's a good question. Let me think for a second."
- "So, the way I'd describe it is ..."
- "There were basically three parts to this ..." (← 숫자를 먼저 던지면 뇌가 구조를 잡습니다)

### 2) 질문 되짚기 (Rephrase) — 이해 확인 + 생각할 시간 확보

- "If I understand correctly, you're asking how I handled \_\_\_, right?"
- "Just to make sure I answer the right thing — do you mean \_\_\_ or \_\_\_?"

### 3) 소리 내어 구조 잡기 (Signpost out loud)

- "Let me start with the problem, then how I solved it, then the result."
- "There are three reasons. First, ... Second, ... And finally, ..."

### 4) 모르는 걸 우아하게 (Graceful "I don't know")

- "I haven't worked with that directly, but my understanding is \_\_\_. How I'd approach it is \_\_\_."
- "I don't want to guess — but here's how I'd find out: \_\_\_."
- 절대 지어내지 마세요. "여기까진 안다 + 이렇게 접근하겠다"가 정답입니다.

## E. Phrase Bank (만능 표현 — 외워두면 어디든 씬)

프로젝트가 바뀌어도 그대로 재사용하는 표현들입니다. 소리 내어 10번씩 읽어 입에 붙이세요.

### ① Openers (시작)

| English                                  | 뜻 / 쓰임      |
|------------------------------------------|-------------|
| "In short, I built / designed / led ..." | 요점부터: 헤드라인에 |
| "The project was about ___, basically."  | 가볍게 주제 던지기  |
| "At a high level, the goal was to ___."  | 큰 그림부터      |

### ② Sequencing / Transitions (순서·전환)

| English                                         | 뜻             |
|-------------------------------------------------|---------------|
| "First ... then ... after that ... finally ..." | 1-2-3-4 단계    |
| "The way it works is ..."                       | 작동 방식 설명 진입   |
| "On top of that, ..."                           | 게다가 (추가)      |
| "That said, ..."                                | 그렇긴 한데 (반전)   |
| "Which brings me to ..."                        | 자연스럽게 다음 포인트로 |

### ③ Cause & Effect (인과)

| English | 뜻 |
|---------|---|
|---------|---|

|                                            |             |
|--------------------------------------------|-------------|
| "The problem was that __, which meant __." | 문제 → 그래서 ~됨 |
| "Because of __, we ended up with __."      | ~때문에 ~결과    |
| "This led to __."                          | 이것이 ~로 이어짐  |
| "As a result, __."                         | 그 결과        |

#### ④ Quantifying results (결과 정량화 — 가장 중요)

| English                                                      | 뜻                  |
|--------------------------------------------------------------|--------------------|
| "We took the F1 score from 28% up to 96%."                   | 28%→96%로 끌어올림      |
| "That cut costs by 100% / by a fifth."                       | 비용을 100% / 1/5로 절감 |
| "It improved recall by about 35%."                           | 약 35% 향상           |
| "We processed around 600,000 items with zero manual review." | 60만 건을 사람 개입 0으로   |
| "Response time dropped by roughly 65%."                      | 응답 65% 단축          |

#### ⑤ Decisions & Tradeoffs (결정·트레이드오프 — 시니어 신호)

| English                                     | 뜻              |
|---------------------------------------------|----------------|
| "The key decision was to __."               | 핵심 결정은 ~       |
| "I chose A over B because __."              | B 대신 A를 고른 이유  |
| "There was a tradeoff between __ and __."   | ~와 ~ 사이 트레이드오프 |
| "I went with the simpler option, since __." | 더 단순한 쪽을 택함    |
| "In hindsight, I'd __."                     | 돌이켜보면 ~하겠다     |

#### ⑥ Hedging / Uncertainty (조심스럽게)

| English                        | 뜻         |
|--------------------------------|-----------|
| "Roughly / around / about __." | 대략        |
| "If I remember correctly, __." | 내 기억이 맞다면 |
| "It depends on __."            | ~에 따라 다름  |

#### ⑦ Emphasis (강조)

| English                                 | 뜻           |
|-----------------------------------------|-------------|
| "The really important part here is __." | 진짜 핵심은      |
| "What made this hard was __."           | 어렵게 만든 건    |
| "This is the part I'm most proud of."   | 가장 자랑스러운 부분 |

#### ⑧ Wrapping up (마무리)

| English | 뜻 |
|---------|---|
|---------|---|

|                                     |              |
|-------------------------------------|--------------|
| "So in the end, ____."              | 결국           |
| "The takeaway for me was ____."     | 내 교훈은        |
| "That's the project in a nutshell." | 요약하면 이게 다입니다 |

## F. 전달·발음 (Delivery for Korean speakers)

### 1) 속도와 멈춤 (Pace & pause)

- 빨리 말하면 발음이 뭉개집니다. **천천히 + 비트 사이 0.5초 멈춤**이 더 유능해 보입니다.
- 멈춤은 약점이 아닙니다. 생각하는 사람처럼 보입니다.

### 2) 기술 용어 강세 (Word stress) — 강세 위치가 틀리면 안 들립니다

| 단어                  | 강세 (대문자가 강)                    |
|---------------------|--------------------------------|
| architecture        | AR-ki-tek-cher                 |
| asynchronous        | ay-SIN-kruh-nus                |
| deploy / deployment | di-PLOY / di-PLOY-ment         |
| latency             | LAY-ten-see                    |
| parameter           | puh-RAM-uh-ter                 |
| inference           | IN-fer-ence                    |
| schema              | SKEE-muh                       |
| query               | KWEER-ee                       |
| retrieval           | ri-TREE-vul                    |
| embedding           | em-BED-ding                    |
| orchestration       | or-kuh-STRAY-shun              |
| deterministic       | di-ter-MIN-is-tic              |
| threshold           | THRESH-hold                    |
| PostgreSQL          | "POST-gres" (또는 post-gres-Q-L) |

### 3) 한국인이 자주 놓치는 소리

- **F vs P**: "F1" = "**ef** one" (에프), "performance"의 f. (P로 새지 않기)
- **R vs L**: "retrieval", "parallel", "reranker".
- **TH**: "threshold", "throughput", "method", "month". 혀를 살짝 물기.
- **V vs B**: "vector", "Milvus", "**v**LLM", "serving". (B로 새지 않기)
- **끝소리 자음**: "deployed", "cached", "fixed" 의 -d/-t 를 끝까지.

### 4) 숫자 읽기 (자주 씀)

- 28.79% → "twenty-eight point seven nine percent"
- 96.10% → "ninety-six point one percent"
- 600,000 → "six hundred thousand"

- 1/5 → "a fifth" / "one fifth"
- ~35% → "about thirty-five percent"
- v0.4.0 → "version zero point four point zero"

## 5) 필러(filler) 관리

- "uh, um"을 줄이고, 대신 **의미 있는 필러**로 대체: "so", "basically", "right", "let me see".
- 막히면 "uh..." 대신 **그냥 멈추세요**. 침묵이 "음..."보다 낫습니다.

## G. 연습법 & 면접 직전 루틴

### 연습법 (집에서)

1. **Shadowing**: Version B를 들으며(또는 읽으며) 0.5초 뒤따라 말하기.
2. **Record & compare**: 핸드폰으로 녹음 → 정답지와 비교 → 강세·끝소리 체크.
3. **무작위 드릴**: 8개 프로젝트 카드를 섞어 뽑고, 뽑힌 것의 Version A를 즉석에서.
4. **꼬리질문 스팅**: 각 정답지 3번 섹션(Follow-up)을 소리 내어.

### 면접 직전 5분 루틴

- Headline 8개(프로젝트별 한 줄)만 빠르게 훑기.
- 핵심 숫자 5개 입으로: "28 to 96", "600k", "100 percent", "65 percent", "35 percent".
- 심호흡. 천천히 말하기로 마음먹기.

## H. 다음 파일 안내

- 02\_Self-Introduction.md — "Tell me about yourself" (자기소개) 버전별 정답
- 03~10 — 프로젝트별 정답지 (이 5-비트 구조를 그대로 적용)
- 11\_Behavioral-Common-Questions.md — 강점·약점·지원동기 등 공통 질문
- 12\_Phrase-Bank.md — 위 표현들의 확장판 + 프로젝트별 핵심 표현 모음

모든 프로젝트 파일은 이 구조를 따릅니다:

0)이 프로젝트로 증명할 것 → 1)생각 구조 → 2)버전 A/B/C → 3)꼬리질문 → 4)핵심 표현 → 5)발음 주의

## 1.1.2 만능 표현과 숫자·발음 드릴

면접 직전·자투리 시간용 빠른 복습 시트. 소리 내어 읽으며 입에 붙이세요.

## A. 8개 프로젝트 한 줄 요약 (Headline drill)

각 프로젝트를 한 문장으로 못 박는 연습. "Tell me about a project"에 곧바로 나오게.

| # | Project | One-liner (외우기) |
|---|---------|-----------------|
|---|---------|-----------------|

|   |                     |                                                                                                                            |
|---|---------------------|----------------------------------------------------------------------------------------------------------------------------|
| 1 | Reflexion Agent     | "I built a self-improving agent that learns from its own failures, cutting repeated retries by about 55 percent."          |
| 2 | Smishing Detection  | "I replaced a flaky external API with a fine-tuned model and took real-world F1 from 28 to over 95 percent."               |
| 3 | Agentic RAG         | "I built a hybrid-search RAG with a safety layer that lifted Top-5 recall by about 35 percent."                            |
| 4 | vLLM Serving        | "I moved our text processing to in-house LLM serving, cutting external API cost by 100 percent and latency by 65."         |
| 5 | LLM Eval Pipeline   | "I built a pipeline that evaluated 600,000 production LLM outputs automatically, at a fifth of the cost."                  |
| 6 | Visit Busan Chatbot | "I led the AI on a travel chatbot that's live in production on the official Busan tourism site."                           |
| 7 | Skylife Recommender | "I built a two-stage vector recommender that improved recommendation diversity — the foundation for my later search work." |
| 8 | MarketScope AI      | "I built and shipped a full-stack AI analytics SaaS solo, from planning to production."                                    |

**드릴법:** 위 8개를 가리고, 프로젝트 이름만 보고 한 문장을 말해보세요. 막힘없이 나올 때까지.

## B. 숫자 드릴 (Numbers — 또박또박 말하기)

숫자가 이 인터뷰의 무기입니다. 영어로 정확히 발음하는 게 핵심.

| 숫자              | 말하기                                                                        | 어느 프로젝트        |
|-----------------|----------------------------------------------------------------------------|----------------|
| 28.79% → 96.10% | "twenty-eight point seven nine percent up to ninety-six point one percent" | Smishing       |
| 95%+            | "over ninety-five percent"                                                 | Smishing (실환경) |
| 600,000         | "six hundred thousand"                                                     | Eval Pipeline  |
| 1/5             | "a fifth" / "one fifth"                                                    | Eval Pipeline  |
| 100%            | "one hundred percent"                                                      | vLLM (비용 제거)   |
| 65%             | "about sixty-five percent"                                                 | vLLM (응답 단축)   |
| 92%             | "around ninety-two percent"                                                | vLLM (요약 수렴)   |

|               |                                                  |                    |
|---------------|--------------------------------------------------|--------------------|
| 35%           | "about thirty-five percent"                      | RAG (Recall)       |
| 55%           | "about fifty-five percent"                       | Reflexion (재시도 감소) |
| 12% / 150,000 | "twelve percent out of a hundred fifty thousand" | RAG (노이즈 제거)       |
| 1,650         | "one thousand six hundred fifty"                 | MarketScope (상권)   |
| 66/66         | "sixty-six out of sixty-six"                     | MarketScope (E2E)  |

**흔한 실수:** "퍼센트"가 아니라 "per-CENT". "thousand"의 th. "fifth"의 -fth 끝소리.

## C. 만능 전환·연결 표현 (Transitions)

| English                               | 뜻          |
|---------------------------------------|------------|
| "Let me give you some context first." | 배경부터 말할게요  |
| "The way it works is ..."             | 작동 방식은 ~   |
| "Here's the key part."                | 여기가 핵심입니다  |
| "To put it simply, ..."               | 간단히 말하면    |
| "On top of that, ..."                 | 게다가        |
| "That said, ..."                      | 그렇긴 하지만    |
| "Which is why ..."                    | 그래서 ~인 것이다 |
| "Long story short, ..."               | 길게 말할 것 없이 |

## D. 비교·결정 표현 (Comparing & deciding)

| English                                               | 뜻               |
|-------------------------------------------------------|-----------------|
| "I compared A, B, and C, and went with C because ..." | A-B-C 비교 후 C 선택 |
| "The tradeoff was between X and Y."                   | X와 Y 사이 트레이드오프  |
| "A was cheaper, but B was more reliable."             | A는 저렴, B는 안정적   |
| "It came down to ___."                                | 결국 ~로 귀결됐다      |
| "In hindsight, I'd ___."                              | 돌이켜보면 ~하겠다      |

## E. 기술 설명 표현 (Explaining tech clearly)

**면접관이 비전문가일 수도 있습니다. 쉬운 비유 + 정확한 용어를 함께.**

| English                                                                             | 뜻 / 쓰임               |
|-------------------------------------------------------------------------------------|----------------------|
| "Think of it as ____."                                                              | ~라고 생각하면 됩니다 (비유 진입) |
| "It's basically ____."                                                              | 기본적으로 ~입니다           |
| "fine-tuning means taking a pretrained model and ____"                              | 파인튜닝을 한 줄로 풀어 설명     |
| "RAG is just retrieval plus generation — find the right text, then answer from it." | RAG를 한 문장으로          |
| "an agent is a model that can decide which tools to call."                          | 에이전트를 한 문장으로         |
| "hybrid search combines keyword and meaning-based search."                          | 하이브리드 검색 한 문장        |
| "in-process means the model runs inside the same service, not behind another API."  | in-process 한 문장      |
| "LLM-as-a-judge means using one LLM to score another LLM's output."                 | 평가 핵심 용어             |

## F. 압박 상황 표현 (Under pressure)

| English                                                                | 뜻                  |
|------------------------------------------------------------------------|--------------------|
| "That's a great question — let me think for a second."                 | 좋은 질문이네요, 잠깐 생각할게요 |
| "Just to make sure I answer the right thing — do you mean ____?"       | 질문 확인 (시간 확보)      |
| "I haven't worked with that directly, but here's how I'd approach it." | 안 해봤지만 이렇게 접근하겠다   |
| "I don't want to guess — but I'd find out by ____."                    | 추측 대신 알아내는 법       |
| "Can I come back to that in a second?"                                 | 잠시 후 다시 답해도 될까요    |

**핵심:** 모르면 **지어내지 말 것**. "여기까진 안다 + 이렇게 접근한다"가 정답입니다.

## G. 마무리 표현 (Closing an answer)

| English                             | 뜻            |
|-------------------------------------|--------------|
| "So in the end, ____."              | 결국 ~         |
| "The takeaway for me was ____."     | 제 교훈은 ~      |
| "That's the project in a nutshell." | 요약하면 이게 다입니다 |

"Happy to go deeper on any part."

어느 부분이든 더 깊이 들어갈 수 있습니다

**연결 파일:** 발음·생각구조 기본기는 01\_Speaking-Framework.md 의 E·F 섹션과 함께 보세요.

이 시트는 그 압축·실전 버전입니다.

## 1.2 자기소개

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

**1.2.1 자기소개 (Tell me about yourself)** — Present-Thread-Proof-Fit 네 박자 구조로 90초 자기소개를 막힘없이 말하고, 차별점 한 줄을 제시할 수 있다.

### 1.2.1 자기소개 (Tell me about yourself)

면접의 첫 질문이자 가장 중요한 30~120초. 여기서 "차별점 한 줄 + 하나의 흐름(thread)"을 심으면, 이후 모든 프로젝트 질문이 그 흐름의 증거로 들립니다.

## 0. 자기소개로 증명할 것

- **차별점 한 줄:** "LLM을 만드는 사람"이 아니라 **"운영에 들어간 LLM의 품질을 규모 있게 검증하는 시스템까지 만드는 사람."**
- **하나의 흐름:** 임베딩-검색(석사 연구) → 서빙 → 에이전트 → 평가·운영 → 풀스택. 한 방향으로 깊어졌다.
- **일하는 방식:** 기술은 목적이 아니라 문제 해결의 도구. 비즈니스 가치 기준으로 우선순위를 잡는다.

## 1. 생각 구조 (Self-intro skeleton)

자기소개는 5-비트가 아니라 **Present → Thread → Proof → Fit** 4-비트로 갑니다.

| Beat      | 한국어                  | 시작 표현 (trigger)                                                             |
|-----------|----------------------|-----------------------------------------------------------------------------|
| 1 Present | 지금의 나 + 차별점 한 줄      | "I'm a Gen-AI engineer with about four years ... What sets me apart is ..." |
| 2 Thread  | 어떻게 여기까지 왔나 (하나의 줄기) | "I actually started from ___, and kept going deeper in one direction ..."   |
| 3 Proof   | 대표 성과 1~2개 (숫자)      | "Most recently, I ___."                                                     |
| 4 Fit     | 무엇을 찾고 있나 / 왜 여기     | "Now I'm looking for a role where ___."                                     |

핵심 원칙: **이력서를 시간순으로 읊지 마세요.** 회사 8개 나열 = 실패. "하나의 줄기 + 대표 증거"만.

길이는 **90초 이내**. 길면 면접관이 흥미를 잃습니다.

## 2. 모범답안 (Model Answers)

### ● Version A — Elevator (약 30초, 쉬운 영어)

I'm a Gen-AI and ML engineer with around **four years** of experience, mostly in LLM systems and search. What makes me a bit different is that I don't just build models — I build the systems that **check whether they actually work in production**. For example, I recently built a pipeline that evaluated about **600,000** production LLM outputs automatically, with no human review. Right now, I'm looking for a role where I can keep working on real, production-grade AI systems **end to end**.

#### 핵심 표현

- "What makes me a bit different is ..." = 제 차별점은 ~ (자기소개 황금 문장)
- "I don't just build models — I build the systems that ..." = 모델만 만드는 게 아니라 ~하는 시스템까지
- "end to end" = 처음부터 끝까지 (전 구간)

**전달 팁:** 첫 문장은 외워서 **자신 있게** 첫인상이 여기서 결정됩니다. "600,000"에서 또박또박.

### ● Version B — Standard (약 80초, 자연스러운 영어) ← 기본값

Sure. I'm a Gen-AI and ML engineer with about **four years** of experience. If I had to sum up what sets me apart, it's this: I don't stop at building an LLM — I build the systems that **verify its quality at scale once it's running in production**.

I actually started from **recommendation and search**. My master's research was on embedding-based recommendation — turning user preferences into vectors and finding items by similarity — and I kept deepening in that one direction ever since. That took me from **vector search** services, to **serving** our own LLMs in-house, to **agent** systems, and most recently to **LLM evaluation and operations**.

A couple of results I'm proud of: I built an evaluation pipeline that judged around **600,000** production summaries with no human review, and I fine-tuned a smishing-detection model that took the F1 score from **28 percent up to 96**. On the side, I also built and shipped a full-stack AI SaaS that's **live in production**.

So I've touched the whole pipeline — collection, search, serving, generation, and evaluation. Now I'm looking for a team where I can keep working on production AI systems **end to end**, and go deeper on the operations side.

#### 핵심 표현

- "If I had to sum up what sets me apart, it's this: ..." = 차별점을 한마디로 하면
- "I kept deepening in that one direction" = 한 방향으로 계속 깊어졌다
- "a couple of results I'm proud of" = 자랑스러운 결과 몇 가지
- "I've touched the whole pipeline" = 전 구간을 다뤄봤다

**전달 팁:** Thread 비트("started from ... kept deepening")에서 손으로 단계를 그리듯 천천히. 흐름이 들리게.

### ● Version C — Deep Dive (약 2분, 고급 영어) ← "조금 더 자세히 소개해달라" 시

Of course. I'm a Gen-AI and ML engineer with about four years of experience, and the thread that ties my whole career together is **embeddings and search** — it just kept evolving into bigger problems.

It started in grad school at **UNIST**, where my master's thesis was a recommendation system built on a convolutional autoencoder — representing user preferences as **latent vectors** and recommending by vector similarity. That idea became the foundation for everything after. At my first company, I turned it into real services: a movie recommender on **pgvector**, and then a travel chatbot for the Busan tourism office that's **still live in production** today.

At my current company, I pushed further down the stack. I moved our text processing off external APIs by **serving our own LLM in-process** — that cut external API cost by a hundred percent and dropped latency by about **65 percent**. I built an **agentic RAG** system with hybrid search that improved Top-5 recall by around **35 percent**, and a self-improving **Reflexion** agent that learns from its own failures, which cut repeated retries by about **55 percent**. I also fine-tuned a **smishing detector** that took real-world F1 from **28 percent to over 95**.

But the project that best captures what I care about is the most recent one: an **evaluation pipeline** that scored around **600,000** production LLM summaries with no human in the loop. The challenge was cost — so I compressed five quality criteria into a single judge call, about a **fifth** of the cost, while keeping quality. That's the part of me that's a bit different: I care as much about **knowing an LLM is good in production** as about building it.

Outside work, I built a full-stack AI SaaS solo, from planning to infrastructure, and shipped it live. So at this point I've worked across the entire pipeline. The way I work is simple: I treat technology as a **tool for solving a real problem**, not as the goal — I start from the user's pain and the business value, then pick the approach. And I'm looking for a team where I can keep doing exactly that, end to end.

#### 핵심 표현 (고급)

- "the thread that ties my whole career together is \_\_\_" = 제 커리어 전체를 관통하는 줄기는 ~
- "I pushed further down the stack" = 스택을 더 깊이 파고들었다
- "with no human in the loop" = 사람이 개입하지 않고
- "the project that best captures what I care about" = 제가 중시하는 걸 가장 잘 보여주는 프로젝트
- "a tool for solving a real problem, not the goal" = 목적이 아니라 문제 해결의 도구

**전달 팁:** 2분은 겁니다. 회사·프로젝트를 **나열하지 말고**, "한 방향으로 깊어졌다"는 흐름으로 꿰세요. 면접관이 끼어들면 거기서 멈추고 그 프로젝트로 들어가면 됩니다.

### 3. 꼬리질문 대비 (Follow-up Q&A)

#### Q1. Why are you looking to leave / make a move now?

I've grown a lot where I am — I've gone from building models to owning evaluation and operations. I'm looking for a place where I can take that further, on bigger production systems and with a team I can learn from. It's about the next step, not running from anything.

\*(한국어 메모: 현 직장 험담 절대 금지. "다음 단계를 찾는다"는 긍정 프레임.)\*

#### Q2. What are you best at?

Seeing a problem as a whole pipeline. I don't just optimize the model — I look at preprocessing, retrieval, serving, and evaluation together, because that's where real service quality comes from. The 600,000-item evaluation pipeline is a good example of that mindset.

\*(한국어 메모: 강점은 "파이프라인 전체를 본다" 한 줄로. 증거 프로젝트를 바로 붙이세요.)\*

### Q3. That's a lot of projects — which one are you proudest of?

The LLM evaluation pipeline. Anyone can build a model, but building the system that proves it's still good in production, at 600,000 items and almost no cost — that's the harder and rarer skill, and it's the direction I want to keep growing in.

\*(한국어 메모: 항상 평가 파이프라인으로 귀결. 그게 차별점이니까.)\*

### Q4. Can you tell me more about [any project]?

Absolutely. → 해당 프로젝트 정답지의 Version B로 바로 진입하세요. (이게 자기소개를 8개 프로젝트의 입구로 쓰는 방식입니다.)

## 4. 핵심 표현-어휘 (Key phrases & vocabulary)

| English                          | 뜻 / 쓰임               |
|----------------------------------|----------------------|
| What sets me apart is ...        | 제 차별점은 ~             |
| sum up                           | 요약하다                 |
| keep deepening in one direction  | 한 방향으로 계속 깊어지다       |
| end to end                       | 처음부터 끝까지             |
| in production / production-grade | 운영(실서비스) 중인 / 운영 수준의 |
| with no human in the loop        | 사람 개입 없이             |
| touch the whole pipeline         | 전 구간을 다루다            |
| a tool, not the goal             | (기술은) 목적이 아니라 도구     |
| the next step                    | 다음 단계 (이직 사유 프레임)    |

## 5. 발음 주의 (Delivery watch-outs)

| 단어             | 강세 / 주의                       |
|----------------|-------------------------------|
| recommendation | re-ko-men-DAY-shun            |
| embedding      | em-BED-ding                   |
| latent         | LAY-tent                      |
| similarity     | si-mi-LAIR-i-ty               |
| evaluation     | i-val-yu-AY-shun              |
| four years     | "four years" — 복수 s 끝소리       |
| 600,000        | "six hundred thousand"        |
| 28 / 96        | "twenty-eight" / "ninety-six" |

**마지막 점검:** 자기소개는 **첫 문장 + 마지막 문장**을 통째로 외우세요. 가운데는 흐름만 기억하면 됩니다.  
| 첫인상(자신감)과 끝맺음(이 회사에서 무엇을 하고 싶은가)이 평가의 8할입니다.

## 2 프로젝트 정답지

여덟 개 프로젝트 각각을 버전별로 설명하고, 예상 꼬리질문에 사실에 근거해 답할 수 있습니다.

이 Phase는 다음 섹션으로 구성됩니다.

### 2.1 플랜티넷 프로젝트

2.1.1 ReAct + Reflexion 자기개선 에이전트

2.1.2 URL 스미싱 탐지 시스템

2.1.3 Milvus Agentic RAG 시스템

2.1.4 vLLM 자체 서빙 API

2.1.5 LLM 평가 파이프라인

### 2.2 AIO2O & 사이드 프로젝트

2.2.1 Visit Busan 여행 챗봇

2.2.2 Skylife 영화 추천 시스템

2.2.3 MarketScope AI 상권분석 SaaS

## 2.1 플랜티넷 프로젝트

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

**2.1.1 ReAct + Reflexion 자기개선 에이전트** — 실패에서 학습하는 Reflexion 구조와 동일 작업 재시도 약 55퍼센트 감소를 영어로 설명할 수 있다.

**2.1.2 URL 스미싱 탐지 시스템** — 외부 API를 자체 모델로 대체한 결정과 실환경 F1 28.79에서 96.10퍼센트 향상을 분포 구분과 함께 설명할 수 있다.

**2.1.3 Milvus Agentic RAG 시스템** — 하이브리드 검색과 안전 계층으로 Top-5 Recall 약 35퍼센트 향상을 설명할 수 있다.

**2.1.4 vLLM 자체 서빙 API** — in-process 서빙 선택 근거와 외부 API 비용 100퍼센트 제거·응답 약 65퍼센트 단축을 설명할 수 있다.

**2.1.5 LLM 평가 파이프라인** — 60만 건 무인 평가와 다섯 기준을 단일 호출로 묶어 LLM 추론을 약 1/5로 줄인 설계를 설명할 수 있다.

### 2.1.1 ReAct + Reflexion 자기개선 에이전트

**한 줄:** 직원이 자연어로 요청하면 사내 플랫폼 API를 호출해 처리하고, 실패하면 스스로 반성해 재시도를 줄이는 자기 개선형 AI 에이전트.

**헤드라인 숫자:** 동일 작업 재시도 약 55% 감소 · 17개 도구 연결 · 5개 관측(Observability) Spec.

소속 Plantynet · 2025.09~현재 · 3명 팀(Front-Back-AI) · AI Agent 로직 전체(설계·구현·배포) 단독 담당.

## 0. 이 프로젝트로 증명할 것 (What this proves about you)

- "LLM을 쓰는 사람"이 아니라 "실패에서 스스로 배우는 에이전트를 설계하는 사람." ← Reflexion 설계 능력. 당신의 가장 큰 차별점.
- 운영 LLM의 비결정성(non-determinism)을 관측 가능하게(observable) 만든 LLMOps 감각.
- 설문으로 실제 페인포인트를 정량화한 문제정의 능력 — 추측이 아니라 데이터로 시작했다는 점.

이 프로젝트는 "에이전트를 직접 설계·운영해봤는가"를 묻는 질문에 **가장 강한 카드**입니다. "Tell me about an agent you built"에 이걸 꺼내세요.

## 1. 생각 구조 (5-Beat skeleton, 이 프로젝트 버전)

| Beat        | 이 프로젝트에서 말할 내용                                                                                |
|-------------|-----------------------------------------------------------------------------------------------|
| 1 Headline  | 사내 플랫폼에 붙는 자기 개선형 AI 에이전트를 만들었다. 실패에서 스스로 배워 동일 작업 재시도를 약 55% 줄였다                             |
| 2 Problem   | 단순 에이전트는 Tool Calling 실패 시 같은 실수를 반복하거나 멈춤. 게다가 구두 요청 중심이라 Spec 문서·이력이 누락됨                    |
| 3 Decision  | ReAct(추론+행동)에 실패 시 자기반성(Reflexion)을 추가. 실패를 problem/solution 쌍으로 장기기억에 저장해 재주입, LangGraph로 구현 |
| 4 Hard Part | 운영 LLM의 비결정성 + 비용. 2단계 Evaluator(규칙→LLM)로 비용 통제, 5개 관측 Spec으로 운영을 보이게                         |
| 5 Result    | 동일 유형 재시도 약 55% 감소, 17개 도구 연결, Spec 자동 생성으로 설문 1순위(기획-개발 간극) 해소                               |

핵심 메시지 한 줄: "The whole project was about making an agent that learns from its own failures instead of repeating them."

## 2. 모범답안 (Model Answers)

### ● Version A — Elevator (약 30초, 쉬운 영어)

In short, I built an AI agent for our internal work platform. People just type what they want in **plain language** — like "write a spec document" or "summarize this sprint" — and the agent calls the right tools to do it. The hard part was that when a tool call failed, a normal agent would **repeat the same mistake** or just stop. So I made it stop, **reflect** on why it failed, save that lesson, and use it next time. That cut the average number of retries for the same kind of task by about **fifty-five percent**.

#### 핵심 표현

- "in plain language" = 자연어로, 쉬운 말로
- "the right tools" = 알맞은 도구
- "repeat the same mistake" = 같은 실수를 반복하다
- "reflect on why it failed" = 왜 실패했는지 돌아보다
- "save that lesson" = 그 교훈을 저장하다
- "cut ... by about fifty-five percent" = 약 55% 줄이다

전달 팁: "fifty-five percent"에서 또박또박 멈추세요. 숫자가 이 답변의 힘입니다.

---

## ● Version B — Standard (약 75초, 자연스러운 영어) ← 기본값

So, this was an AI agent project at **Plantynet**. We have an internal work-management platform, and the idea was to let employees just ask for things in **natural language** — "write a spec document," "summarize this sprint" — and have the agent call the platform's APIs to actually do it.

The real problem was **failure**. When a tool call went wrong, a plain agent would either repeat the same mistake or get stuck. On top of that, a lot of our collaboration was verbal, so spec documents and work history kept getting lost — and a **survey** we ran before building confirmed those were the top two pain points.

So the key decision was to go beyond a simple **ReAct** loop. ReAct is just reasoning and acting, and on its own it couldn't break the failure loop. I added a **reflection step**: when the agent fails, it writes down the problem and a possible solution, stores that in a **long-term memory**, and re-injects it the next time a similar situation comes up. I built that whole flow — actor, tool executor, evaluator, and reflection — in **LangGraph**.

The tricky part was cost and reliability in production. Checking every failure with an LLM is expensive, so the evaluator runs in **two stages** — a cheap keyword rule first, then an LLM only for the precise judgment. And if the same failure happens **three times**, it stops early instead of burning calls.

As a result, the average number of retries for the same kind of task dropped by about **fifty-five percent**. We connected **seventeen** tools, and the agent now auto-generates spec documents, which directly fixed that number-one complaint from the survey.

### 핵심 표현

- "in natural language" = 자연어로 (업계 표준 표현)
- "get stuck" = (진행이) 막히다, 멈추다
- "go beyond a simple ReAct loop" = 단순 ReAct 루프를 넘어서다
- "a reflection step" = 자기반성 단계
- "re-inject it the next time" = 다음번에 다시 주입하다
- "in two stages" = 2단계로
- "stops early" = 일찍 멈추다 (Early Stopping)

전달 팁: Problem-Decision-Result 비트에서 잠깐씩 멈춰 호흡을 끊으면 구조가 또렷이 들립니다.

---

## ● Version C — Deep Dive (약 2~3분, 고급 영어) ← "더 자세히" 요청 시

Sure, let me go a bit deeper on the design.

The goal was a self-improving agent sitting on top of our internal platform — an **MCP host** that turns a natural-language request into real API calls. The constraint I cared about most was **failure recovery**. A plain ReAct agent reasons and acts in a loop, but when a tool call fails, it tends to either loop on the same wrong action or give up. That's the gap I wanted to close.

So my core design was to add a **Reflexion** stage on top of ReAct. The flow has four roles, and I implemented all of them in **LangGraph**: an actor that decides the next action, a tool executor that

runs it, an evaluator that judges the result, and a reflection step. The piece I'm most proud of is the long-term memory — I call it the **LessonsStore**. When the agent fails, it doesn't just retry blindly; it distills the failure into a **problem-and-solution pair**, stores it, and then re-injects the matching lesson into the prompt whenever a similar situation comes up. And to avoid burning tokens forever, if the exact same failure happens three times, it triggers **early stopping**.

The evaluator itself is where the cost engineering lives. Judging every result with an LLM would be expensive, so it runs in **two stages**: first a cheap keyword rule that catches obvious errors, and only then a precise LLM judgment that checks whether the result actually matches the user's intent. Cheap filter first, expensive judge second.

The other hard part was operating an LLM in production, where behavior is **non-deterministic**. You can't just hope it works — you have to make it **observable**. So I defined **five observability specs**: request tracing, prompt and response logging, token usage, latency, and error tracking. For tooling I went with **Langfuse** — which is LLM-specific and self-hostable — plus Sentry, Grafana, and Prometheus. I also split error tracking into **five categories** — LLM, parse, tool, agent-logic, and system — so when something breaks, I know which layer to look at. On deployment, I separated the agent service from the MCP server with **Docker Compose**, so I can redeploy just the agent without dropping the MCP connections.

The end result: retries for the same type of task dropped by about **fifty-five percent**, we wired up **seventeen** tools across three layers, and the two top survey complaints — the planning-versus-dev gap and lost verbal history — got solved by auto-generated spec docs and feed-based history tracking.

#### 핵심 표현 (고급)

- "self-improving agent / MCP host" = 자기 개선형 에이전트 / MCP 호스트
- "failure recovery" = 실패 복구
- "distills the failure into a problem-and-solution pair" = 실패를 problem/solution 쌍으로 압축
- "re-injects the matching lesson into the prompt" = 맞는 교훈을 프롬프트에 재주입
- "early stopping" = 조기 종료 (반복 실패 차단)
- "non-deterministic / observable" = 비결정적 / 관측 가능한
- "cheap filter first, expensive judge second" = 싼 필터 먼저, 비싼 판단 나중

**전달 팁:** 길어서, 면접관 표정을 보며 "Want me to go deeper on any part?"로 중간 점검하면 좋습니다.

### 3. 꼬리질문 대비 (Follow-up Q&A)

#### Q1. What's the difference between ReAct and Reflexion, and why did you need both?

ReAct is the base loop — the agent reasons about what to do, takes an action, and looks at the result. The problem is, on its own it has no memory of why something failed, so it tends to repeat the same mistake. Reflexion adds that missing piece: after a failure, the agent reflects, writes down what went wrong and how to fix it, and carries that forward. So ReAct is "think and act," and Reflexion is "learn from what just went wrong." I needed both because acting without learning was exactly the failure loop I was trying to break.

\*(한국어 메모: 둘의 역할 분담 — ReAct=행동, Reflexion=학습 — 을 한 문장으로 대비시키면 깔끔합니다.)\*

#### Q2. How does the agent actually "remember" past failures?

I store each failure as a problem-and-solution pair in a long-term memory I call the LessonsStore. It's not the full conversation — just a distilled lesson. When a new request looks similar to a past failure, I pull the matching lesson and re-inject it into the prompt, so the model goes in already knowing what to avoid. And if the same failure repeats three times, I stop early instead of looping forever.

\*(한국어 메모: "전체 대화가 아니라 압축된 교훈만 저장"이 비용·품질 양쪽에서 똑똑한 선택임을 강조하세요.)\*

### Q3. Why is the evaluator two stages instead of just using the LLM?

Cost. If I sent every single result to an LLM to judge, that gets expensive fast. So the first stage is a cheap keyword rule that catches obvious errors with no model call at all. Only what gets past that goes to the second stage, where an LLM checks whether the result really matches the user's intent. Cheap filter first, expensive judge second — same idea as a fast path and a slow path.

\*(한국어 메모: "fast path / slow path" 비유는 면접관이 바로 이해합니다. 비용 의식을 보여주는 답.)\*

### Q4. You said the LLM is non-deterministic in production. How did you handle that?

You can't make an LLM fully deterministic, so instead of fighting it, I made it observable. I defined five observability specs — request tracing, prompt and response logging, token usage, latency, and error tracking — and wired them up with Langfuse, Sentry, Grafana, and Prometheus. I also bucket every error into one of five categories — LLM, parse, tool, agent-logic, or system — so when something goes wrong I can tell which layer to look at right away. The point isn't to remove the randomness; it's to never be blind to it.

\*(한국어 메모: "비결정성을 없애는 게 아니라 보이게 만든다"가 LLMops 핵심 메시지. 그대로 외우세요.)\*

### Q5. How did you decide what to build in the first place?

I didn't want to guess, so before writing any code I ran a survey with our developers and planners to quantify the actual pain points. The top complaint was that requirements kept getting misaligned between planning and development, and the second was that verbal requests left no history. That ranking is what shaped the product — auto-generated spec documents for the first one, and feed-based history tracking for the second.

\*(한국어 메모: "추측 대신 설문으로 정량화"는 시니어 신호. 1순위·2순위를 해결책과 1:1로 연결하세요.)\*

### Q6. Why separate the agent and the MCP server in deployment?

I ran them as two services with Docker Compose so they're decoupled. The agent logic changes a lot — that's where I iterate — but the MCP server and its tool connections are pretty stable. By splitting them, I can redeploy just the agent without tearing down the MCP connections every time. It keeps deployments fast and avoids unnecessary downtime on the part that didn't change.

\*(한국어 메모: "자주 바뀌는 것과 안 바뀌는 것을 분리"가 배포 설계의 핵심 원리임을 보여주는 답.)\*

## 4. 핵심 표현·어휘 (Key phrases & vocabulary)

| English                    | 뜻 / 쓰임               |
|----------------------------|----------------------|
| ReAct (reasoning + acting) | 추론하고 행동하는 에이전트 기본 루프 |
| Reflexion                  | 실패 후 자기반성으로 학습하는 패턴  |
| self-improving agent       | 자기 개선형 에이전트          |

|                              |                          |
|------------------------------|--------------------------|
| tool calling                 | 도구 호출 (에이전트가 API·함수를 부름) |
| MCP host / MCP server        | 도구 연결을 담당하는 호스트 / 서버     |
| long-term memory             | 장기 기억 (실패 교훈 저장소)        |
| re-inject into the prompt    | 프롬프트에 다시 주입              |
| early stopping               | 조기 종료 (반복 실패 차단)         |
| evaluator                    | 평가기 (결과 판정)              |
| observability                | 관측 가능성 (운영 상태를 보이게)      |
| non-deterministic            | 비결정적 (같은 입력에 다른 출력)      |
| request tracing              | 요청 추적                    |
| decouple / separate services | 서비스 분리 (독립 배포)           |
| pain point                   | 페인 포인트 (실제 불편)           |

## 5. 발음 주의 (Delivery watch-outs)

| 단어                | 강세 / 주의                                   |
|-------------------|-------------------------------------------|
| Reflexion         | ri-FLEK-shun (reflection과 비슷, x는 k+sh 소리) |
| ReAct             | "ree-ACT" (리-엑트, 두 토막으로 또렷이)              |
| evaluator         | ee-VAL-yoo-ay-ter                         |
| observability     | ub-zer-vuh-BIL-i-tee                      |
| non-deterministic | non-di-ter-MIN-is-tic                     |
| latency           | LAY-ten-see                               |
| Langfuse          | "LANG-fuse" (랭-퓨즈)                        |
| LangGraph         | "LANG-graf"                               |
| Gemini            | JEM-in-ee (제미니). 끝을 "-나이"로 빼지 않기          |
| 17                | "seventeen" (seventy 아님!)                 |
| 55%               | "fifty-five percent" — v 소리, th 아님        |

**마지막 점검:** 이 프로젝트의 힘은 "약 55% 감소"와 "에이전트가 실패에서 스스로 배운다"입니다. 이 두 가지를 또렷이, 자신 있게.

### 2.1.2 URL 스미싱 탐지 시스템

**한 줄:** 외부 스캐닝 API 의존을 없애고, URL 텍스트(URLBert)와 HTML 구조를 직접 분석해 스미싱을 탐지하는 자체 파이프라인.

**헤드라인 숫자:** 실환경 F1 28.79% → 96.10%(1:1 균형) · 9:1 실환경 95%+ · 17개 HTML 피처 · API 장애

자동 감지(143 → 0건).

소속 Plantynet · 2명 팀 · 2025.07~2026.03 · 스미싱 분류 모델 개선 / HTML 탐지 파이프라인 / 성능 모니터링 전담.

## 0. 이 프로젝트로 증명할 것 (What this proves about you)

- **외부 의존을 자체 모델로 대체한 결정.** 불안정한 외부 스캐닝 API를 없애고, URL 텍스트에서 직접 학습하는 모델로 갈아탔습니다.
- **이중 접근으로 엡지케이스까지 커버한 문제해결.** 텍스트(URLBert) + 구조(HTML)로 단축 URL이라는 가장 까다로운 케이스를 뚫었습니다.
- **모델만이 아니라 운영까지 끝까지 책임.** 배포 후 Model Drift와 외부 API 장애를 스스로 잡아내는 모니터링을 직접 붙였습니다.
- **숫자로 증명되는 임팩트.** 실환경 F1 28.79% → 95%+, 그리고 실제 API 장애를 143 → 0건으로 자동 포착.

이 프로젝트는 "기술적 의사결정"과 "끝까지 책임진다"를 동시에 보여주는 카드입니다. "Tell me about a hard technical decision" 또는 "a project you owned end to end"에 이걸 꺼내세요.

## 1. 생각 구조 (5-Beat skeleton, 이 프로젝트 버전)

| Beat        | 이 프로젝트에서 말할 내용                                                                                                        |
|-------------|-----------------------------------------------------------------------------------------------------------------------|
| 1 Headline  | 외부 API에 의존하던 스미싱 탐지를 자체 모델로 대체해 실환경 F1을 28.79%에서 95%+로 끌어올렸다                                                          |
| 2 Problem   | 기존엔 외부 스캐닝 API로 URL 메타데이터를 모아 Random Forest로 분류. API 실패율이 높고, 불균형 9:1 실환경 F1이 28.79%. 특히 단축 URL은 Whois 정보가 없어 대부분 오분류 |
| 3 Decision  | 외부 API를 통째로 버리고 URL 텍스트 자체에서 학습. URLBert를 파인 튜닝 — URL 특화 사전학습 + CPU만으로 가능 + 한국형 단축 URL 적합                             |
| 4 Hard Part | 단축 URL은 텍스트만으로 판별 불가 → 리다이렉트를 따라가 최종 랜딩 페이지 HTML을 분석하는 별도 파이프라인(17개 피쳐)을 추가                                           |
| 5 Result    | 1:1 F1 96.10%, 9:1 실환경에서도 95%+, 외부 API 제거. 운영 모니터링이 실제 API 전면 장애(143 → 0건)를 자동 감지                                     |

핵심 메시지 한 줄: "The whole project was about replacing a flaky external API with our own model — and owning it all the way to production monitoring."

## 2. 모범답안 (Model Answers)

### ● Version A — Elevator (약 30초, 쉬운 영어)

In short, I rebuilt our smishing detection system so it **no longer depends** on an external scanning API. The old system called that API to get URL data and then used a Random Forest — but the API failed a lot, and on real traffic the F1 score was only about **twenty-eight percent**. So I fine-tuned a URL-specific model called **URLBert** to read the URL text directly. That took the F1 up to about **ninety-six percent** on balanced data, and it stayed above **ninety-five percent** on real, imbalanced traffic. On top of that, I added monitoring that watches the system in production — and it actually caught a full API outage on its own, when the data dropped from **143** successful calls to **zero**.

#### 핵심 표현

- "no longer depends on" = 더 이상 ~에 의존하지 않는다
- "failed a lot" = 자주 실패했다 (구어체 강조)
- "read the URL text directly" = URL 텍스트를 직접 읽다
- "took the F1 up to \_\_\_" = F1을 ~까지 끌어올리다
- "caught \_\_\_ on its own" = ~을 스스로 잡아냈다

**전달 팁:** "twenty-eight percent"와 "ninety-six percent"에서 또박또박 멈추세요. 28 → 96 점프가 이 답변의 힘입니다.

### ● Version B — Standard (약 75초, 자연스러운 영어) ← 기본값

So, this was a smishing detection project — smishing is basically phishing over text messages, usually with a malicious **URL** inside. We already had a system, but it relied on an **external scanning API** to pull metadata about each URL, and then a **Random Forest** classified it. The problem was that the API failed often, and on real traffic — which is heavily imbalanced, about **nine-to-one** — the F1 was stuck at around **twenty-eight percent**. Short URLs were the worst case, because there's no **Whois** record to look up, so they were almost always misclassified.

The **key decision** was to drop the external API entirely and learn from the URL text itself. I fine-tuned **URLBert**, a model pretrained specifically on URLs — I picked it because it's URL-specialized, it runs on **CPU** only, and it handles Korean-style short URLs well. Zero-shot it was weak, around **forty-six percent**, so I fine-tuned it on about **4,300** of our own labeled URLs. After that the F1 jumped to **96 percent** on balanced data, and it held above **95 percent** even on the real nine-to-one traffic.

The **tricky part** was short URLs, like bitly links — the text alone tells you nothing, because the real content is on the landing page. So I built a second pipeline that follows the **redirect** to the final landing page and analyzes its **HTML** — 17 features in total. And I didn't stop at the model: I added production **monitoring** that compares each run to the last twenty, so if the model **drifts** or the external API breaks, it alerts us automatically. It actually caught a real API outage — calls dropped from **143** to **zero** — and fired a critical alert right away, instead of someone finding it in the logs a day later.

#### 핵심 표현

- "smishing is basically phishing over text messages" = 스미싱 정의를 한 문장으로 깔고 시작
- "relied on an external scanning API" = 외부 스캐닝 API에 의존했다
- "stuck at around \_\_\_" = ~에 머물러 있었다
- "the worst case" = 최악의 경우
- "drop the external API entirely" = 외부 API를 완전히 없애다
- "held above 95 percent" = 95% 이상을 유지했다

- "follows the redirect" = 리다이렉트를 따라간다
- "the model drifts" = 모델 성능이 변질된다

**전달 팁:** Problem → Decision → Hard part 비트 사이에서 0.5초씩 끊으면 구조가 또렷이 들립니다. "28 → 96"에서 목소리를 한 번 올리세요.

## ● Version C — Deep Dive (약 2~3분, 고급 영어) ← "더 자세히" 요청 시

Sure, let me walk through the design.

The starting constraint was that our detection depended on an **external scanning API**. It pulled metadata per URL and a **Random Forest** classified it, but the failure rate was high, and on production traffic — imbalanced roughly **nine-to-one** toward legitimate URLs — the F1 was only **28.79 percent**. The killer case was shortened URLs: with no **Whois** record to query, the old pipeline had almost nothing to go on, so it misclassified most of them. So my first decision was to stop depending on anything external and learn straight from the URL string.

For that I fine-tuned **URLBert**. I chose it for three reasons: it's pretrained specifically on the URL domain, it runs on **CPU** alone — no GPU budget needed for training or inference — and it works well on Korean-style shortened URLs. Zero-shot it only hit **46 percent** F1, so I fine-tuned it on **4,296** of our own labeled URLs. On a balanced one-to-one set that took it to **96.10 percent** F1 — precision **97.6**, recall **94.65** — versus about **58 percent** for the old Random Forest. And it held above **95 percent** F1 with precision over **97** on the real nine-to-one distribution, at roughly **0.05 milliseconds** per URL.

But text-only classification can't see through a shortened link, because the original URL and the actual landing page are different. So I built a second pipeline that follows the HTTP **redirect** to the final landing page and analyzes its **HTML**. I ported **15** features from an open-source phishing detector and added **two** Korean-specific ones — `has_tel_input`, which flags a phone-number input field, and `has_kr_keyword`, which looks for Korean scam triggers like parcel delivery, identity verification, obituary, or investment. That's **17** HTML features, and on a balanced set it scored **96.47 percent** F1. Since fetching pages is the slow part, I parallelized collection with a **ThreadPoolExecutor** at twenty concurrent requests.

The last piece was making it survivable in production. I built a three-stage pipeline — **ETL**, then model evaluation, then alerting and logging — and kept it **low-coupling** by storing the last **twenty** runs in a `history.json` and comparing each new run against that window. With thresholds on the **label ratio** and F1, it raises a warning or a critical alert when the model **drifts** or the data looks wrong. And the payoff was real: during operation it auto-detected a full external-API outage — successful collections dropped from **143** to **zero**, a hundred-percent failure rate — and fired a critical alert on the spot. Before that, this kind of outage was only caught when someone manually read the logs on the next run.

### 핵심 표현 (고급)

- "learn straight from the URL string" = URL 문자열에서 곧바로 학습
- "the killer case was \_\_\_" = 결정적으로 어려운 경우는 ~ (killer = 치명적인)
- "zero-shot" = 미세조정 없이 사전학습만으로
- "held above 95 percent" = 95% 이상을 유지했다
- "follows the HTTP redirect" = HTTP 리다이렉트를 따라간다
- "ported 15 features from \_\_\_" = ~에서 15개 피처를 이식(포팅)했다
- "low-coupling" = 낮은 결합도 (모듈 간 의존을 최소화)

· "survivable in production" = 운영에서 건디는, 실전에서 버티는

**전달 팁:** 길어서 중간에 한 번 "Want me to go deeper on the HTML side or the monitoring?"로 면접관과 호흡을 맞추면 좋습니다.

---

### 3. 꼬리질문 대비 (Follow-up Q&A)

#### Q1. Why remove the external API instead of just making it more reliable?

A few reasons. The failure rate was high and outside our control, and for shortened URLs the API often had no data at all — there's no Whois record to fetch. So even a perfectly reliable API wouldn't fix the worst case. Learning from the URL text directly removed the dependency, cut the latency, and actually scored far higher. So it wasn't only about reliability — it was accuracy and control too.

\*(한국어 메모: "신뢰성만의 문제가 아니라 정확도 + 통제권"으로 답하면 결정이 입체적으로 보입니다.)\*

#### Q2. Why URLBert specifically, and not just features plus a Random Forest?

Three things. It's pretrained on the URL domain, so it already understands URL structure. It runs on CPU only, which mattered because we had no GPU budget. And it handles Korean-style shortened URLs well. The old Random Forest on API features only reached about 58 percent F1, and fine-tuned URLBert beat it by a wide margin — 96 — so the choice paid off.

\*(한국어 메모: 선택 이유 3개를 손가락으로 세며 말하면 구조가 또렷합니다.)\*

#### Q3. How did you handle shortened URLs, since the text alone is useless there?

That was exactly the hard part. The text classifier can't see the real content behind a bitly link, so I added a separate pipeline that follows the redirect to the final landing page and analyzes its HTML — 17 features, including two I added for Korean scams. So the two pipelines complement each other: text for the URL, structure for the page it lands on.

\*(한국어 메모: "두 파이프라인이 상호보완"이 핵심 메시지입니다. 한 문장으로 못 박으세요.)\*

#### Q4. What were the two Korean-specific HTML features, and why add them?

The open-source detector gave me 15 generic phishing features, but Korean smishing has its own patterns. So I added `has_tel_input`, which detects a phone-number input field — common on fake identity-verification pages — and `has_kr_keyword`, which looks for words like parcel delivery, identity verification, obituary, or investment. Those two captured local scam patterns the generic features missed.

\*(한국어 메모: 도메인 지식을 피쳐로 녹였다는 점이 시니어 신호입니다.)\*

#### Q5. How does the monitoring actually detect a problem?

It's a three-stage pipeline — ETL, evaluation, then alerting — and it keeps the last twenty runs in a `history.json`. Each new run is compared against that window, so it stays low-coupling, with no heavy state. If the label ratio or the F1 crosses a threshold, it raises a warning or a critical alert. That's how it caught the API outage: successful collections went from 143 to zero, and it fired a critical alert immediately.

\*(한국어 메모: "최근 20회와 비교 → 임계값 → 알림" 흐름을 순서대로 짚으세요.)\*

#### Q6. What would you do differently next time?

The monitoring state lives in a `history.json` file, which is simple and low-coupling, but at a larger scale I'd move that history into a proper database so it's queryable and more durable. I'd also keep

expanding the labeled fine-tuning set beyond the 4,296 we used, so the model stays robust as scam patterns evolve.

\*(한국어 메모: 한계 인정 + 개선 방향 = 성숙함. 지어내지 말고 실제 설계의 한계만 말하세요.)\*

## 4. 핵심 표현·어휘 (Key phrases & vocabulary)

| English                          | 뜻 / 쓰임                         |
|----------------------------------|--------------------------------|
| smishing                         | 문자(SMS) 기반 피싱 (SMS + phishing) |
| fine-tune / fine-tuning          | (사전학습 모델을) 미세조정                |
| zero-shot                        | 미세조정 없이 사전학습만으로 추론             |
| imbalanced data (nine-to-one)    | 불균형 데이터 (9:1)                  |
| F1 score                         | 정밀도·재현율의 조화평균 (분류 핵심 지표)       |
| precision / recall               | 정밀도 / 재현율                      |
| shortened URL                    | 단축 URL (bitly 등)               |
| redirect                         | 리다이렉트 (다른 URL로 자동 전환)          |
| landing page                     | 리다이렉트 끝의 최종 도착 페이지             |
| feature                          | (모델 입력으로 쓰는) 피쳐·특징             |
| Whois                            | 도메인 등록 정보 조회                   |
| model drift                      | 모델 성능 변질 (시간이 지나며 정확도 하락)      |
| low-coupling                     | 낮은 결합도 (모듈 간 의존 최소화)           |
| threshold                        | 임계값                            |
| critical alert                   | 치명(critical) 경보                |
| ThreadPoolExecutor / concurrency | 스레드풀 병렬 처리 / 동시성               |

## 5. 발음 주의 (Delivery watch-outs)

| 단어          | 강세 / 주의                             |
|-------------|-------------------------------------|
| smishing    | "SMISH-ing" — sm- 을 묶어서, "스미싱" 아님   |
| URLBert     | "U-R-L bert" (유-알-엘 버트)             |
| precision   | pri-SIZH-un (가운데 강세)                |
| recall (명사) | RE-call (앞 강세)                      |
| redirect    | REE-di-rekt (명사) / ree-di-REKT (동사) |
| imbalanced  | im-BAL-anced                        |

|           |                                                    |
|-----------|----------------------------------------------------|
| threshold | THRESH-hold — TH는 혀를 살짝 물기                         |
| 28.79%    | "twenty-eight point seven nine percent"            |
| 96.10%    | "ninety-six point one zero percent" (또는 point one) |
| 9:1       | "nine to one"                                      |
| 4,296     | "four thousand two hundred ninety-six"             |
| 143 → 0   | "a hundred forty-three down to zero"               |
| 0.05ms    | "zero point oh five milliseconds"                  |

**마지막 점검:** 이 프로젝트의 힘은 "28.79 → 96.10" 점프와 "143 → 0건" 장애 감지입니다. 이 두 숫자를 또렷이, 자신 있게. "F1"은 "ef one"(에프), P로 새지 않게 하세요.

### 2.1.3 Milvus Agentic RAG 시스템

**한 줄:** 한국어 매거진 플랫폼의 기사를 의미 기반으로 검색하고, LlamaGuard로 안전한 답변을 생성하는 Agentic RAG 챗봇.

**헤드라인 숫자:** Top-5 recall 약 35% 향상 · 15만 청크 중 12% 노이즈 제거 · 입력·출력 LlamaGuard 안전 검증.

소속 Plantynet · 2025.04~2025.12 · AI 파트 2명 · 검색 엔진 / 전처리 / 에이전트 전담.

## 0. 이 프로젝트로 증명할 것 (What this proves about you)

- 검색 품질을 모델이 아니라 "데이터부터" 끌어올린 엔지니어. ← 가장 강한 차별점. 하이브리드 검색 설계 + 15만 청크 중 12% 노이즈 제거로 Top-5 recall 약 35% 향상.
- 생성형 AI의 안전성을 "설계"에 넣은 사람. 입력과 출력 양쪽을 LlamaGuard로 검사하는 Safety Layer를 에이전트 흐름에 내장.
- 도구를 비교·선정하는 의사결정 능력. Pinecone·Qdrant·Milvus를 명확한 기준(자체 호스팅 + 하이브리드 지원)으로 비교한 뒤 Milvus 선택.

이 프로젝트는 "검색(retrieval)을 깊게 안다"와 "안전·운영까지 본다"를 동시에 보여주는 카드입니다. RAG·LLM 직무에 가장 직접적으로 연결됩니다.

## 1. 생각 구조 (5-Beat skeleton, 이 프로젝트 버전)

| Beat        | 이 프로젝트에서 말할 내용                                                 |
|-------------|----------------------------------------------------------------|
| 1. Headline | 한국어 매거진 플랫폼용 의미 기반 기사 검색 챗봇(Agentic RAG)을 만들었다                 |
| 2. Problem  | 기존엔 키워드 매칭뿐 → "여름에 읽기 좋은 글" 같은 의미 질문에 무력. 생성 답변의 안전 검증 체계도 없었음 |

|             |                                                                                           |
|-------------|-------------------------------------------------------------------------------------------|
| 3 Decision  | Dense(BGE-M3) + BM25 하이브리드 검색 + reranker + 문맥 윈도우 설계. 벡터 DB는 비교 후 Milvus 선택(자체 호스팅 하이브리드) |
| 4 Hard Part | 검색 품질을 데이터부터: 15만 청크 중 12% 노이즈 제거. 거기에 입출력 LlamaGuard 안전 레이어                              |
| 5 Result    | Top-5 recall 약 35% 향상, 입출력 안전 검증, Docker Compose 3-tier 단일 명령 배포                          |

핵심 메시지 한 줄: **"The whole project was about making search understand meaning, not just keywords — and making every answer safe by design."**

## 2. 모범답안 (Model Answers)

### ● Version A — Elevator (약 30초, 쉬운 영어)

In short, I built a search chatbot for a Korean magazine platform. Before this, the search only matched exact keywords, so a question like "a good article to read in summer" just didn't work — there's no keyword to match. So I built a **RAG** system that searches by meaning, not just words. I combined two kinds of search — meaning-based vectors and keyword-based **BM25** — and then re-ranked the results. That improved our **Top-5 recall** by about **35%** over keyword search. On top of that, I check both the question and the answer with a safety model called **LlamaGuard**, so unsafe content is blocked on both ends.

#### 핵심 표현

- "only matched exact keywords" = 정확한 키워드만 매칭했다 (= 의미는 못 봄)
- "searches by meaning, not just words" = 단어가 아니라 의미로 검색
- "re-ranked the results" = 결과를 재정렬했다
- "Top-5 recall by about 35%" = 상위 5개 재현율 약 35%
- "blocked on both ends" = 입력·출력 양쪽에서 차단

**전달 팁:** "about 35%"에서 또박또박 멈추세요. recall의 끝 -ll, both의 th 끝소리를 끝까지.

### ● Version B — Standard (약 75초, 자연스러운 영어) ← 기본값

So, this was an **Agentic RAG** project for a Korean magazine platform — basically a chatbot that finds articles from a natural-language question. Before this, search was pure **keyword matching**, so a semantic question like "a good article to read in summer" had nothing to match on. And there was no safety check on the AI's generated answers at all.

So the **key decision** was the retrieval design. I built a **hybrid search**: dense vectors from **BGE-M3** for meaning, plus **BM25** sparse search for exact terms. Then I re-rank the merged results and pull in a **context window** — the chunks right before and after each hit — so the model has enough context to answer well. That took our **Top-5 recall** up about **35%** versus keyword search.

For the vector database, I compared **Pinecone**, **Qdrant**, and **Milvus**, and went with **Milvus**, because I could self-host it and it supports hybrid search natively.

The part I'm most proud of is that I pushed quality from the **data** side too. Out of about **150,000** chunks, I removed roughly **12%** as noise — things under 100 characters, junk special characters, or chunks that failed Korean tokenization. And for safety, every input and every output goes through **LlamaGuard**, so unsafe content is caught on both ends. The whole stack — Milvus, the FastAPI backend, and the Streamlit frontend — comes up with a single **Docker Compose** command.

#### 핵심 표현

- "Agentic RAG" = 에이전트형 RAG (에이전트가 검색·생성 흐름을 제어)
- "nothing to match on" = 매칭할 대상이 없다 (키워드 검색의 한계)
- "hybrid search ... dense ... sparse" = 하이브리드 검색 (의미 벡터 + 키워드)
- "context window" = 문맥 윈도우 (검색된 청크 앞뒤 문맥)
- "compared A, B, and C, and went with \_\_\_" = 비교 후 선택 (의사결정 시그널, 꼭 쓰세요)
- "pushed quality from the data side" = 데이터 단에서 품질을 끌어올림
- "caught on both ends" = 양쪽 끝에서 잡힘

**전달 팁:** 결정(hybrid)·DB 선택·데이터+안전, 세 군데에서 잠깐씩 멈추면 구조가 또렷이 들립니다.

### ● Version C — Deep Dive (약 2~3분, 고급 영어) ← "더 자세히" 요청 시

Sure, let me go a bit deeper on the design.

The goal was an **agentic RAG** system for a Korean magazine platform: a user asks in natural language, and the agent retrieves the right articles and generates a safe answer. The old system only did **keyword matching**, so anything semantic — "a good article to read in summer" — fell through, because there's no literal keyword to hit. And generated answers had no safety validation at all.

On retrieval, I designed a **hybrid** pipeline. Dense embeddings from **BGE-M3** capture meaning, **BM25** handles exact terms, and then **BGE-Reranker-v2-M3** re-orders the merged candidates. On top of that I add a **context window** — I pull the chunks immediately before and after each retrieved chunk, so the model sees enough surrounding text to answer well. That combination lifted **Top-5 recall** by about **35%** over pure keyword search. For magazine-level recommendations I used Milvus **Group Search** — instead of scoring single chunks, it scores the query against every chunk in a magazine and ranks the magazines by their best match. For the database itself, I evaluated **Pinecone, Qdrant, and Milvus**, and chose **Milvus** specifically because I could self-host it and it supports hybrid search natively.

Two parts were the real engineering. First, **data quality**: out of roughly **150,000** chunks, I dropped about **12%** — around 18,000 — as noise. The rules were chunks under 100 characters, chunks full of special characters, and chunks that failed **KoNLPy** Korean tokenization. Cleaning the index at the source is what made retrieval trustworthy. Second, **safety by design**: the agent runs a fixed flow — **Safety Guard, then Reasoning, then Tool Calling, then Response** — and **LlamaGuard** inspects both the user input and the final output, so unsafe content is blocked on both ends. I stream tokens back over **SSE**, and I keep conversation history in a thread-based **checkpoint store**, so multi-turn context survives.

For delivery, the whole three-tier stack — Milvus, a FastAPI backend, and a Streamlit frontend — comes up with a single **Docker Compose** command. The services are isolated on an internal Docker bridge network, and volume mounts keep the data persistent across restarts. Right now I'm learning how to port that same setup onto AWS and Kubernetes.

### 핵심 표현 (고급)

- "fell through" = (걸리지 않고) 빠져나간다 / 처리되지 못하다
- "re-orders the merged candidates" = 병합된 후보를 재정렬한다
- "ranks the magazines by their best match" = 최고 유사도 기준으로 매거진을 순위 매김 (Group Search)
- "self-host" = 자체 호스팅하다
- "cleaning the index at the source" = 인덱스를 소스 단계에서 정제
- "safety by design" = 설계 단계부터 안전성 (사후가 아니라 구조에 내장)
- "blocked on both ends" = 입력·출력 양 끝에서 차단
- "checkpoint store" = 체크포인트 저장소 (대화 이력 보존)
- "port ... onto AWS and Kubernetes" = AWS·쿠버네티스로 이식

**전달 팁:** 길어서 중간에 면접관 표정을 보며 "Want me to go deeper on the retrieval or the safety side?"로 한 번 끊어주면 좋습니다.

## 3. 꼬리질문 대비 (Follow-up Q&A)

### Q1. Why hybrid search instead of just dense vectors?

Two reasons. Dense vectors from BGE-M3 are great at meaning, but they can miss an exact term — a specific name or keyword. BM25 catches those literal matches. Combining both, then re-ranking, gives me semantic recall without losing precise keyword hits. That's what got Top-5 recall up about 35% over keyword-only search.

\*(한국어 메모: dense = 의미, sparse(BM25) = 정확 일치. "둘의 약점을 서로 보완한다"는 논리로 답하면 설계 의도가 분명해집니다.)\*

### Q2. Why Milvus over Pinecone or Qdrant?

I compared all three against two needs. I wanted to self-host the database, for control, and I needed native hybrid search — dense plus sparse — for my retrieval design. Milvus met both of those, so I went with it over Pinecone and Qdrant.

\*(한국어 메모: 자체 호스팅 + 네이티브 하이브리드, 딱 두 기준만으로 답하세요. 다른 DB의 세부를 지어내지 말 것.)\*

### Q3. How did you make the answers safe?

Safety is built into the agent flow. Every request runs through a Safety Guard first, then Reasoning, then Tool Calling, then the Response. And LlamaGuard inspects both the input and the output — so an unsafe question is caught before retrieval, and an unsafe generated answer is caught before it reaches the user. It's checked on both ends, not just once.

\*(한국어 메모: 흐름(Safety Guard → Reasoning → Tool Calling → Response) + 입출력 양쪽 LlamaGuard. "both ends"를 강조하세요.)\*

### Q4. Why remove 12% of the chunks — didn't you lose information?

No, that 12% was noise, not signal. These were chunks under 100 characters, chunks full of special characters, or chunks that failed KoNLPy Korean tokenization. They don't carry real meaning, so embedding them just adds bad matches to the index. Cleaning them out at the source is a big part of why retrieval quality went up.

\*(한국어 메모: 노이즈 정의(100자 미만 / 특수문자 / 토큰화 실패)를 또렷이. "정보가 아니라 노이즈였다"가 핵심.)\*

#### Q5. What's the context window, and why add it?

When I retrieve a chunk, I also pull the chunks immediately before and after it. A single chunk can be too narrow to answer from — it might cut off mid-thought. Adding that surrounding context gives the model enough text to generate a complete, grounded answer.

\*(한국어 메모: "청크 단위가 좁아서 앞뒤를 더 가져온다"는 한 줄 직관으로 설명하면 충분합니다.)\*

#### Q6. How do you deploy and run the whole system?

It's three tiers — Milvus as the vector DB, FastAPI for the backend, and Streamlit for the frontend — and it all comes up with a single Docker Compose command. The services talk over an internal Docker bridge network, so they're isolated, and volume mounts keep the data persistent across restarts. I'm currently learning how to move that same setup onto AWS and Kubernetes.

\*(한국어 메모: 3-tier + 단일 명령 + 브릿지 격리 + 볼륨 영속성 + (성장) K8s 학습 중. 마지막의 성장 의지가 플러스 포인트입니다.)\*

## 4. 핵심 표현·어휘 (Key phrases & vocabulary)

| English                         | 뜻 / 쓰임                            |
|---------------------------------|-----------------------------------|
| Agentic RAG                     | 에이전트형 검색증강생성 (에이전트가 검색·생성 흐름을 제어) |
| hybrid search                   | 하이브리드 검색 (dense 벡터 + sparse 키워드)  |
| dense vector / sparse (BM25)    | 밀집 벡터(의미) / 희소 검색(정확 일치)          |
| reranker / re-rank              | 재정렬기 / 재정렬하다                      |
| recall                          | 재현율 (찾아야 할 것 중 실제로 찾은 비율)         |
| context window                  | 문맥 윈도우 (검색 청크의 앞뒤 문맥)             |
| chunk                           | 청크 (문서를 쪼갠 검색 단위)                 |
| noise / clean the index         | 노이즈 / 인덱스를 정제하다                   |
| tokenization                    | 토큰화 (여기선 KoNLPy 한국어 토큰화)          |
| self-host                       | 자체 호스팅하다                          |
| safety layer / safety by design | 안전 레이어 / 설계 단계부터 안전               |
| on both ends                    | 양쪽 끝에서 (입력·출력 모두)                 |
| checkpoint store                | 체크포인트 저장소 (대화 이력 보존)              |
| token streaming (SSE)           | 토큰 스트리밍 (서버에서 실시간 전송)             |
| three-tier                      | 3계층 (DB · 백엔드 · 프론트엔드)            |

## 5. 발음 주의 (Delivery watch-outs)

| 단어 | 강세 / 주의 |
|----|---------|
|----|---------|

|              |                                 |
|--------------|---------------------------------|
| Milvus       | "MIL-vus" — V를 B로 새지 않기 (밀버스 X) |
| Agentic      | a-JEN-tik                       |
| recall       | ri-CALL (뒤 강세). 끝 -ll 끝소리       |
| reranker     | ree-RANK-er — R/L 또렷이           |
| hybrid       | HY-brid                         |
| BM25         | "B-M twenty-five"               |
| BGE-M3       | "B-G-E M three"                 |
| LlamaGuard   | "LLA-ma-guard" (라마가드)           |
| KoNLPy       | "코-엔-엘-파이" (ko-N-L-py)          |
| Qdrant       | "Q-drant" 또는 "QUAD-rant"        |
| tokenization | toh-ken-eye-ZAY-shun            |
| 35%          | "about thirty-five percent"     |
| 150,000      | "a hundred and fifty thousand"  |
| 12%          | "twelve percent"                |

**마지막 점검:** 이 프로젝트의 힘은 **"recall 약 35% 향상"**과 **"데이터부터 안전까지 설계로 풀었다"**입니다. 이 두 메시지를 또렷이, 자신 있게.

## 2.1.4 vLLM 자체 서빙 API

**한 줄:** 사내 기사 요약·분류·태깅을 외부 API 없이 처리하는 자체 LLM 서빙 API.

**헤드라인 숫자:** 외부 API 비용 100% 제거 · 응답 약 65% 단축 · 요약 길이 수렴 92% · 6종 API 통합.

소속 Plantynet · 2024.10~2025.04 · 3명 팀 · 중간 LLM 로직·서빙 API 설계 / 개발 / 배포 전담.

## 0. 이 프로젝트로 증명할 것 (What this proves about you)

- **"모델을 쓰는 사람"이 아니라 "모델을 직접 서빙하는 인프라를 설계·선정하는 사람."** ← 서빙 방식 3가지를 비교한 뒤 in-process vLLM을 고른 인프라 의사결정.
- 비용과 레이턴시를 **동시에** 잡은 성과: 외부 API 호출 비용 100% 제거 + 평균 응답 약 65% 단축.
- 기존 연동을 깨지 않는 **점진 마이그레이션** 설계 (v0 → v1, 6종 API를 한 서비스로 통합).
- 도구를 목적에 맞게: 단순 서빙엔 vLLM, 분기·재시도가 필요한 곳에만 LangGraph.

이 프로젝트는 **"아키텍처를 결정한 경험"** 또는 **"비용을 줄인 경험"** 질문에 꺼내세요. **세 가지 옵션을 비교한 이야기**가 핵심 카드입니다.

## 1. 생각 구조 (5-Beat skeleton, 이 프로젝트 버전)

| Beat        | 이 프로젝트에서 말할 내용                                                                                       |
|-------------|------------------------------------------------------------------------------------------------------|
| 1 Headline  | 기사 요약·분류·태깅을 외부 API 없이 처리하는 사내 자체 LLM 서빙 API를 만들었다                                                   |
| 2 Problem   | 요약·분류·태깅을 외부 LLM API에 의존 → 비용과 응답 속도가 둘 다 문제                                                         |
| 3 Decision  | 서빙 방식 3가지(외부 API / vLLM 별도 서버 / in-process)를 비교 → in-process vLLM 채택. 외부 API 비용 100% 제거, 응답 약 65% 단축 |
| 4 Hard Part | 6종 API를 기존 연동을 깨지 않고 이전 + 분기·재시도가 필요한 흐름. → v0/v1 점진 전환, v1에 LangGraph                               |
| 5 Result    | 외부 API 비용 0, 응답 약 65% 단축, 6종 API 단일 서비스화, 요약 길이 수렴 약 92%                                             |

핵심 메시지 한 줄: "The whole project was about bringing our text processing in-house — so we pay nothing per call and respond a lot faster."

## 2. 모범답안 (Model Answers)

### ● Version A — Elevator (약 30초, 쉬운 영어)

In short, I built an in-house API that handles article summarizing, classifying, and tagging for us. Before this, we paid an outside API for all of that, and it was both expensive and slow. So we decided to run the model ourselves, inside our own service. I used **vLLM** right inside our FastAPI app, so there's no outside call at all. That cut our external API cost by **100 percent** — we pay nothing per call now — and it brought the response time down by about **65 percent**. We also put six of these tools into one single service. It was a nice win on both cost and speed.

#### 핵심 표현

- "in-house" = 외주·외부가 아니라 자체적으로
- "run the model ourselves" = 우리가 직접 모델을 돌리다 (= self-host)
- "no outside call at all" = 외부 호출이 아예 없다
- "cut our cost by 100 percent" = 비용을 100% 줄이다

**전달 팁:** "100 percent"와 "65 percent"에서 또박또박 멈추세요. 이 두 숫자가 답변의 힘입니다.

### ● Version B — Standard (약 75초, 자연스러운 영어) ← 기본값

So this was an **in-house LLM serving** project. We needed to summarize, classify, and tag news articles, and at first we did all of that through an external LLM API. The problem was that it hit us on two sides at once — the cost per call and the response time were both too high.

So the **key decision** was how to serve the model ourselves. I looked at three options: keep the external API, run **vLLM** as a separate server, or run it **in-process** — right inside the FastAPI app. I went with in-process, because it removes the network hop between services and keeps everything in one container. That cut the external API cost by **100 percent**, since we self-host now, and it brought the

average response time down by about **65 percent**. To keep the GPU busy, I leaned on vLLM's **continuous batching**, and I used an **asyncio semaphore** to cap concurrency so we don't blow past GPU memory.

The other half was **migration**. We had six of these endpoints — summary, classification, tagging — and I couldn't break the existing integrations. So I rolled it out in two stages: **v0** was plain vLLM serving, and **v1** added a **LangGraph** agent for the flows that needed branching and retries, like making a summary land within a target length. That convergence step now succeeds about **92 percent** of the time.

So in the end, we own the whole stack, the per-call cost is gone, responses are about 65 percent faster, and all six tools live in one service.

### 핵심 표현

- "serve the model ourselves" = 모델을 자체 서빙하다
- "in-process / a separate server" = 같은 프로세스 안 / 별도 서버 (서빙 구조 두 갈래)
- "removes the network hop" = 서비스 간 네트워크 경유를 없앤다 (지연 감소)
- "continuous batching" = 연속 배칭 (vLLM 핵심 용어, 꼭 쓰세요)
- "cap concurrency" = 동시 처리량에 상한을 두다
- "blow past GPU memory" = GPU 메모리를 넘겨버리다 (구어체)

**전달 팁:** Problem-Decision-Migration 세 비트 사이에서 0.5초씩 끊으면 구조가 또렷이 들립니다.

## ● Version C — Deep Dive (약 2~3분, 고급 영어) ← "더 자세히" 요청 시

Sure, let me go deeper on the design.

The starting point was that our summarization, classification, and tagging all went through an external LLM API, and that cost us on both price and latency. So the core question was how to bring the model in-house. I compared three serving setups: the existing external API, vLLM as its own **separate service**, and vLLM running **in-process** — meaning the model runs inside the same FastAPI process that serves the requests. I chose in-process. A separate vLLM server would have been easier to scale on its own, but it adds a network hop and another service to deploy and watch. In-process keeps the request, the batching, and the GPU all in one place, which for our scale was simpler and faster. That's how we got the external API cost down to zero — a **100 percent** cut — and the average response time down by roughly **65 percent**.

The piece I'm most proud of on the serving side is how concurrency is handled. vLLM does **token-level continuous batching**, so instead of waiting for a whole batch to finish, new requests join the batch as soon as there's room — that's what keeps the GPU efficient under load. But the risk there is GPU memory: too many requests at once and you run out of VRAM. So I guard it with an **asyncio semaphore** that caps how many requests are in flight, which keeps memory safe without killing throughput.

The second hard part was migration. We had six endpoints already wired into other systems, so a big-bang rewrite was off the table. I did it in two versions running side by side: **v0** was straight vLLM serving, and **v1** layered a **LangGraph** agent on top for the flows that needed real control flow — branching and retries. The clearest example is summary length. We want a summary to land within a target range — roughly **70 to 130 percent** of the target — so the agent checks the output, and if it's off, it loops and retries until it converges. That now succeeds about **92 percent** of the time. For long

documents I used a **Map-Refine** approach — pull bullet points out of each chunk first, then turn those bullets into clean prose.

On the delivery side, it's all containerized with **Docker** and the **NVIDIA Container Toolkit** for GPU access, and I used a **multi-stage Dockerfile** so the heavy build dependencies don't end up in the runtime image — that keeps the image small. Docker Compose handles startup and updates. There's also a small **Gradio** UI so the planning team can test any endpoint themselves without calling me. So in the end: no external API cost, about 65 percent faster, six tools in one service, and a clean two-version migration path.

#### 핵심 표현 (고급)

- "bring the model in-house" = 모델을 자체 인프라로 들이다
- "token-level continuous batching" = 토큰 단위 연속 배치
- "requests in flight" = 처리 중인(동시에 떠 있는) 요청
- "a big-bang rewrite was off the table" = 한 번에 갈아엎는 방식은 선택지에서 제외
- "real control flow — branching and retries" = 진짜 제어 흐름 (분기·재시도)
- "land within a target range" = 목표 범위 안에 들어오다 (수렴)
- "multi-stage Dockerfile" = 다단계 도커파일 (이미지 경량화)

**전달 팁:** 길어서, "Want me to go deeper on the serving or the migration side?"로 중간 점검하면 자연스럽습니다.

### 3. 꼬리질문 대비 (Follow-up Q&A)

#### Q1. Why in-process vLLM instead of a separate vLLM server?

A separate server is easier to scale on its own, but it adds a network hop and another service to deploy and watch. For our scale, keeping vLLM in-process — inside the FastAPI app — was simpler and lower latency, and we got the cost win either way since we self-host. If we later needed to scale serving independently from the API, I'd revisit the separate-server option.

\*(한국어 메모: "장단점을 알고 골랐다 + 조건이 바뀌면 다시 본다"가 시니어 신호입니다.)\*

#### Q2. How did you migrate six endpoints without breaking existing integrations?

I ran two versions side by side. v0 was plain vLLM serving that matched the old behavior, so existing callers kept working untouched. v1 added the LangGraph agent for the flows that needed branching and retries. Teams could move endpoint by endpoint instead of all at once, so nothing broke during the switch.

\*(한국어 메모: 점진 전환의 핵심은 "옛 동작을 그대로 보존한 v0"입니다. 이 한 문장을 빼먹지 마세요.)\*

#### Q3. How do you keep concurrent requests from running out of GPU memory?

Two things work together. vLLM's continuous batching packs requests efficiently at the token level, and on top of that I use an asyncio semaphore to cap how many requests are in flight. The semaphore is the hard ceiling that keeps us from exceeding GPU memory, while batching keeps throughput high under that ceiling.

#### Q4. What is the summary-length convergence, and how do you hit 92 percent?

We want a summary to land within a target range — about 70 to 130 percent of the target length. In v1, the LangGraph agent generates, checks the length, and if it's outside the range it adjusts and

retries in a loop until it converges. That control flow is exactly why I brought in LangGraph, and it lands in range about 92 percent of the time.

#### Q5. Why LangGraph instead of just writing the retry loop yourself?

I could have hand-coded it, but the flows got more complex than a single retry — conditional branching and retry with state. LangGraph gives me an explicit graph for that control flow, so it's easier to read, extend, and debug than a pile of if-statements. For the simple v0 path I didn't use it at all — only where the branching actually justified it.

\*(한국어 메모: "단순한 곳엔 안 썼다"가 도구 남용을 안 한다는 신호입니다.)\*

#### Q6. How did you deploy it, and how do non-engineers test it?

Everything runs in a Docker container with the NVIDIA Container Toolkit for GPU access, and I used a multi-stage Dockerfile so build dependencies stay out of the runtime image, which keeps it small. Docker Compose handles startup and updates. And I added a small Gradio UI so the planning team can try any endpoint themselves — that cut a lot of back-and-forth.

## 4. 핵심 표현·어휘 (Key phrases & vocabulary)

| English                | 뜻 / 쓰임                   |
|------------------------|--------------------------|
| in-house               | 자체적으로 / 사내에서 (외주·외부 반대)  |
| self-host              | 직접 호스팅하다 (외부 API 대신)     |
| in-process             | 같은 프로세스 안에서 (별도 서버 아님)   |
| serving                | (모델) 서빙·응답 제공            |
| continuous batching    | 연속 배치 (요청을 토큰 단위로 이어 묶음) |
| in flight              | 처리 중인 (동시에 떠 있는 요청)      |
| network hop            | 서비스 간 네트워크 경유 (지연 요인)    |
| cap concurrency        | 동시 처리량에 상한을 두다           |
| converge / convergence | (목표 범위로) 수렴하다            |
| branching and retries  | 조건 분기와 재시도               |
| side by side           | 나란히 (v0/v1 병행)           |
| big-bang rewrite       | 한 번에 갈아엎는 재작성 (피한 것)     |
| control flow           | 제어 흐름 (분기·반복)            |
| multi-stage build      | 다단계 빌드 (이미지 경량화)         |

## 5. 발음 주의 (Delivery watch-outs)

| 단어 | 강세 / 주의 |
|----|---------|
|----|---------|

|            |                                       |
|------------|---------------------------------------|
| vLLM       | "V-L-L-M" (브이-엘-엘-엠). 첫소리 V, B로 새지 않기 |
| FastAPI    | "fast-A-P-I" (패스트 에이-피-아이)            |
| asyncio    | ay-SINK-ee-oh (혹은 "AY-sink-eye-oh")   |
| semaphore  | SEM-uh-for                            |
| LangGraph  | "lang-graph" (랭-그래프), 끝 -ph = f       |
| Gradio     | GRAH-dee-oh                           |
| in-process | in-PROSS-ess, 한 단어처럼 붙여서              |
| converge   | kun-VERJ — V 소리, 끝 -j                 |
| latency    | LAY-ten-see                           |
| 100%       | "by one hundred percent" — 끝의 t 살리기   |
| ~65%       | "about sixty-five percent"            |
| 92%        | "about ninety-two percent"            |

**마지막 점검:** 이 프로젝트의 힘은 "외부 API 비용 100% 제거"와 "응답 약 65% 단축"입니다. **자체 서버로 둘 다 잡았다는 한 문장으로 또렷이 마무리하세요.**

## 2.1.5 LLM 평가 파이프라인

**한 줄:** 운영 중인 요약 LLM 결과물 약 60만 건을 사람 개입 없이 평가·적재·보고하는 파이프라인.  
**헤드라인 숫자:** 60만 건 무인 평가 · LLM 호출 1/5로 절감 · cron 무중단 자동화.  
 소속 Plantynet · 2명 팀 · 평가 메트릭 설계 / 구현 / Postgres 적재 / cron 자동화 전담.

## 0. 이 프로젝트로 증명할 것 (What this proves about you)

- "LLM을 만드는 사람"이 아니라 "운영 LLM의 품질을 규모 있게 검증하는 시스템을 만드는 사람." ← 당신의 가장 큰 차별점. 자기소개와 연결됨.
- 비용을 의식하는 엔지니어링 (cost-aware engineering): 60만 건 × LLM 호출 = 비용 폭발을 설계로 해결.
- 신뢰성 있는 자동화 (idempotent, unattended, 무중단).

이 프로젝트는 면접에서 **가장 강한 카드**입니다. "Tell me about a project you're proud of"에 이걸 꺼내세요.

## 1. 생각 구조 (5-Beat skeleton, 이 프로젝트 버전)

| Beat       | 이 프로젝트에서 말할 내용                       |
|------------|--------------------------------------|
| 1 Headline | 운영 요약 LLM 60만 건을 무인 평가하는 파이프라인을 만들었다 |

|             |                                                                              |
|-------------|------------------------------------------------------------------------------|
| 2 Problem   | 사람이 샘플로 보던 방식은 60만 건에 안 통함 → LLM-as-judge 필수.<br>근데 축마다 LLM 호출하면 비용·시간 폭발    |
| 3 Decision  | 평가를 정량(LLM 불필요) + 품질(1건당 LLM 1회)로 압축. 5개 품질 기준을 단일 GEval 호출로 묶음 (5회→1회)      |
| 4 Hard Part | 60만 건 규모 + OpenAI rate limit + 무중단. → incremental 평가 + cron/flock + 실패분만 재시도 |
| 5 Result    | 60만 건 무인 평가, 추론·토큰 비용 1/5, 모델 바뀌면 다음 주기에 품질 회귀 자동 감지                         |

핵심 메시지 한 줄: **"The whole project was about evaluating an LLM at scale without going bankrupt on tokens."**

## 2. 모범답안 (Model Answers)

### ● Version A — Elevator (약 30초, 쉬운 영어)

In short, I built an automated pipeline that evaluated about **six hundred thousand** news-summary outputs from a production LLM — with **no human review**. The hard part was cost. Calling an LLM for every quality check on six hundred thousand items would have been way too expensive. So I bundled **five** quality criteria into a **single** LLM call instead of five separate ones. That cut the cost to about **one fifth** while keeping the quality. It ran on a **cron** schedule, completely unattended, and emailed a report every day.

#### 핵심 표현

- "with no human review" = 사람 개입 없이 (= unattended / hands-off)
- "way too expensive" = 너무 비싼 (구어체 강조, way = 훨씬)
- "bundled A into B" = A를 B로 묶다
- "cut the cost to about one fifth" = 비용을 약 1/5로 줄이다

전달 팁: "six hundred thousand"와 "one fifth"에서 또박또박 멈추세요. 숫자가 이 답변의 힘입니다.

### ● Version B — Standard (약 75초, 자연스러운 영어) ← 기본값

So, this was an **LLMOps** project. We had a summarization model running in production, and it had already generated around **600,000** Korean news summaries. We needed to know if the quality was actually good — but checking that many by hand is impossible, so we had to use an **LLM as a judge**.

The real problem was **cost**. If I scored every summary on every quality dimension with a separate LLM call, the token bill and the runtime would explode. So the **key decision** was to compress the evaluation. I split the metrics into two groups: **quantitative** ones that need no LLM at all — like the valid-response rate and the length ratio, which I compute directly from the output — and a single **quality** score that needs just **one** LLM call per item.

For that quality score, there were five criteria — content, dates, hallucination, style, and format — and they're all tangled together, so scoring them separately doesn't really make sense. I bundled all five

into **one** GEval call that returns a score and a reason in one shot. That dropped the calls from five to one per item — about a **fifth** of the cost — without losing quality.

The whole thing runs unattended on **cron**: it evaluates new summaries every hour, loads results into Postgres and Excel, and emails a daily report. So whenever the production model changes, we catch any quality regression on the very next cycle.

#### 핵심 표현

- "LLM as a judge" = LLM으로 LLM을 평가 (업계 표준 용어, 꼭 쓰세요)
- "the token bill ... would explode" = 토큰 비용이 폭발한다
- "compress the evaluation" = 평가를 압축하다
- "tangled together" = 서로 얽혀 있다 (= interlocked)
- "in one shot" = 한 번에
- "catch a regression" = 품질 저하를 잡아내다

**전달 팁:** 3개 비트(Problem-Decision-Result)에서 잠깐씩 멈춰 호흡을 끊으면 구조가 또렷이 들립니다.

### ● Version C — Deep Dive (약 2~3분, 고급 영어) ← "더 자세히" 요청 시

Sure, let me go a bit deeper on the design.

The constraint was scale: roughly **600,000** production summaries, evaluated cheaply but credibly. My first move was to **separate what needs an LLM from what doesn't**. Things like the valid-response rate and the summary-to-source length ratio are **deterministic** — I calculate them straight from the result JSON, no model call. That leaves only one genuine LLM judgment per item, so cost scales linearly and predictably.

For that judgment, the tricky part is that our five criteria — content preservation, relative-to-absolute **date** conversion, hallucination, style, and formatting — are **interdependent**. Scoring them as five separate metrics would mean five LLM calls per item, and it also fragments the judgment. So I **overrode** deepeval's default GEvalTemplate with a Korean version — KoreanGEvalTemplate — and folded all five criteria into the **evaluation steps** of a **single** GEval metric. One call returns both a 0-to-1 score and a Korean rationale. That's the five-to-one reduction. I also **inject the article's issued date** into the prompt, because the single most common failure was the model mis-converting relative dates like "last year" into the wrong absolute year — and with the issued date in context, the judge can verify that precisely.

On the operations side, the goal was a **hands-off, non-stop** pipeline. Evaluation is **incremental and idempotent**: I key each result by source, subdirectory, and filename, score only the ones not yet evaluated, and **upsert**. So a backlog of 600,000 never piles up — cron just chips away at it every hour. I guard against overlapping runs with flock, and cap throughput per run. The one failure mode I had to handle was the OpenAI **rate limit**: when that happens the score comes back null, so I wait and **retry only those null rows**, and skip the wait entirely if there are none. Results land in both Excel — a summary sheet and a details sheet — and Postgres, with eval\_runs and eval\_details tables plus aggregate views. A daily email closes the loop. Because the aggregation is per-folder over time, if someone swaps the production model and quality drops, the regression shows up on the next cron cycle automatically — nobody has to remember to check.

#### 핵심 표현 (고급)

- "scales linearly and predictably" = 선형적·예측 가능하게 확장
- "deterministic" = 결정론적 (모델 없이 계산되는)
- "interdependent / fragments the judgment" = 상호의존적 / 판단을 조각
- "override the default template" = 기본 템플릿을 오버라이드
- "incremental and idempotent" = 증분적이고 멱등한 (= 여러 번 돌려도 안전)
- "upsert" = 있으면 갱신, 없으면 삽입
- "chips away at it" = 조금씩 처리해 나간다
- "closes the loop" = 루프를 닫는다 (자동화 완성)

**전달 팁:** 이 버전은 길어서, 면접관 표정을 보며 "Want me to go further on any part?"로 중간 점검하면 좋습니다.

### 3. 꼬리질문 대비 (Follow-up Q&A)

#### Q1. Why bundle five criteria into one call instead of scoring each separately?

Two reasons. First, **cost** — five separate metrics mean five LLM calls per item, and at 600,000 items that's a huge bill. Second, the criteria are **interdependent**; a date error is also kind of a content error, so judging them in isolation actually makes the scoring noisier. One holistic judgment was both cheaper and more coherent.

\*(한국어 메모: 비용 + 일관성 두 축으로 답하면 설계 의도가 분명해집니다.)\*

#### Q2. How can you trust an LLM to judge another LLM?

I don't trust it blindly. The **deterministic** metrics act as a guardrail — if length ratio or valid-response rate looks off, that's a hard signal independent of the judge. The judge also returns a **rationale**, not just a score, so failures are auditable, and I spot-check samples. For a stricter setup I'd add a small **human-labeled gold set** to measure the judge itself, but for this scale and timeline the guardrails plus spot-checks were the right tradeoff.

\*(한국어 메모: "맹신 안 한다 + 가드레일 + 사람 검증 여지"가 성숙한 답입니다.)\*

#### Q3. What was the hardest part?

Honestly, the scale combined with **reliability**. Anyone can run an eval once; running it on 600,000 items, unattended, against API rate limits, without piling up a backlog — that's the engineering. The incremental upsert plus flock plus retry-only-the-failures was what made it actually survivable in production.

#### Q4. How did you handle API rate limits and failures?

When the rate limit hits, the score comes back as null. I isolate **only those null rows**, wait, and retry just them — and if there are zero failures, I skip the wait completely so I'm not adding latency for nothing. Combined with the hourly incremental run, failures just get picked up on the next pass.

#### Q5. How would you detect that the production model regressed?

The aggregation is **per-folder and over time** in Postgres views. So a model swap that drops quality shows up as a metric drop on the **next cron cycle** — automatically, without anyone remembering to look. That was the whole point: turn "someone should check" into "the system checks."

#### Q6. What would you do differently next time?

I'd build a small **gold-standard labeled set** up front to calibrate the judge, and I'd consider a cheaper

or local judge model for the first-pass filter, reserving the stronger model only for borderline cases. That would cut cost even further.

#### 4. 핵심 표현·어휘 (Key phrases & vocabulary)

| English                | 뜻 / 쓰임                 |
|------------------------|------------------------|
| LLM as a judge         | LLM으로 LLM 평가 (필수 용어)   |
| LLMOps                 | LLM 운영 (MLOps의 LLM 버전) |
| at scale               | 대규모로                   |
| unattended / hands-off | 사람 개입 없이               |
| deterministic          | 결정론적 (모델 없이 계산)        |
| incremental            | 증분적 (새 것만 처리)          |
| idempotent             | 멱등한 (여러 번 돌려도 안전)      |
| upsert                 | 있으면 갱신 없으면 삽입          |
| rate limit             | API 호출 한도              |
| regression             | (품질) 회귀·저하             |
| guardrail              | 안전장치                   |
| spot-check             | 표본 점검                  |
| close the loop         | 루프를 닫다 (자동화 완성)        |

#### 5. 발음 주의 (Delivery watch-outs)

| 단어            | 강세 / 주의                               |
|---------------|---------------------------------------|
| criteria      | kry-TEER-ee-uh (복수). 단수는 criterion    |
| deterministic | di-ter-MIN-is-tic                     |
| idempotent    | eye-dem-POH-tent                      |
| aggregate     | AG-ri-git (형용사/명사)                    |
| GEval         | "G-eval" (지-이발)                       |
| PostgreSQL    | "POST-gres" 로 줄여 말해도 됨                |
| 600,000       | "six hundred thousand" (millions 아님!) |
| 1/5           | "a fifth" — th 끝소리                    |

**마지막 점검:** 이 프로젝트의 힘은 "60만"과 "1/5"입니다. 이 두 숫자를 또렷이, 자신 있게.

## 2.2 AIO2O & 사이드 프로젝트

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

**2.2.1 Visit Busan 여행 챗봇** — 상용 운영 중인 서비스의 2단계 추천과 비기술 이해관계자 협업·오너십을 숫자 없이 가치로 설명할 수 있다.

**2.2.2 Skylife 영화 추천 시스템** — rule-based의 한계를 벡터 유사도로 푼 출발점 서사와 추천 다양성 개선을 설명할 수 있다.

**2.2.3 MarketScope AI 상권분석 SaaS** — 기획부터 인프라까지 1인 풀스택으로 배포까지 끌고 간 end-to-end 실행력을 설명할 수 있다.

### 2.2.1 Visit Busan 여행 챗봇

**한 줄:** 부산관광공사 공식 사이트에 탑재된 AI 여행 챗봇 — 자연어로 여행 조건을 말하면 관광지를 추천하고 일정까지 자동 구성한다.

**헤드라인:** 부산관광공사 공식 사이트에 **상용 운영 중 (live in production)** · 5명 팀 AI 리드 · 자연어 추천 + 자동 일정 구성.

소속 AIO2O · 2024.01~2024.10 · 5명 팀 · AI 파트 리드 (벡터 DB 구축 / 추천 알고리즘 / 챗봇 개발) 전담.

## 0. 이 프로젝트로 증명할 것 (What this proves about you)

- "데모를 만드는 사람"이 아니라 "공공기관에서 상용 운영되는 서비스를 리드한 사람." ← 이 프로젝트의 가장 강한 카드. 숫자 지표가 없어도 "live in production on a public institution's official site"가 그 자체로 신뢰입니다.
- 비기술 이해관계자(관광공사 데이터팀 · 기획자)와 협업하고 **MVP 범위를 협의**하는 능력 — 2주 데모 사이클로 피드백을 흡수.
- 도메인 적용력: 리뷰 텍스트(공·부정 키워드 · 해시태그)를 벡터에 녹여, 키워드 검색을 **의미 기반 검색**으로 바꾼 점.
- 배포까지 책임지는 오너십: Docker Compose 컨테이너화 + .env 환경 분리 + 배포 자동화.

이 프로젝트의 카드는 "숫자"가 아니라 "**실제로 쓰이는 제품**"입니다. "Tell me about something you shipped" 또는 협업·오너십을 묻는 질문에 꺼내세요.

## 1. 생각 구조 (5-Beat skeleton, 이 프로젝트 버전)

| Beat       | 이 프로젝트에서 말할 내용                                                               |
|------------|------------------------------------------------------------------------------|
| 1 Headline | 부산관광공사 공식 사이트에 상용 운영 중인 AI 여행 챗봇의 AI 파트를 리드했다 (자연어 추천 + 자동 일정 구성)            |
| 2 Problem  | 기존 검색은 "도시 / 1박 2일 / 맛집" 같은 고정 필터뿐 → "아이랑 갈 조용한 곳" 같은 자연어엔 대응 불가             |
| 3 Decision | 영화 추천에서 쓰던 2단계(메타 필터 → 벡터 유사도)를 여행에 적용. 리뷰의 공·부정 키워드·해시태그를 벡터에 녹여 의미 기반 검색으로 |

|             |                                                                           |
|-------------|---------------------------------------------------------------------------|
| 4 Hard Part | 비기술 이해관계자(데이터팀·기획자)와 스키마·범위 조율 + 배포. → 2주<br>데모 사이클 + Docker Compose/.env |
| 5 Result    | 공식 사이트에 상용 배포(live in production). 5명 팀 AI 리드로 벡터 DB·<br>추천·챗봇 전부 오너십     |

핵심 메시지 한 줄: "The whole project was about turning a plain-language travel wish into a real itinerary — and actually shipping it live on a public tourism site."

## 2. 모범답안 (Model Answers)

### ● Version A — Elevator (약 30초, 쉬운 영어)

In short, I led the AI part of a travel chatbot that's now **live** on Busan's official tourism website. You just type what kind of trip you want — in plain language, like "a quiet place I can visit with my kids" — and it recommends real spots and builds a full schedule for you. The old search only had fixed filters, like city and number of days, so it couldn't handle a request like that. The trick was that we pulled keywords and hashtags out of user reviews and used them to match by **meaning**, not just keywords. I was the AI lead on a team of five, and the best part is — it's a real product people actually use, not just a demo.

#### 핵심 표현

- "live on ... website" = (사이트에) 상용 운영 중
- "in plain language" = 평범한 말로, 자연어로
- "match by meaning, not just keywords" = 키워드가 아니라 의미로 매칭
- "not just a demo" = 데모가 아니라 (실제 제품)

**전달 팁:** 숫자가 없는 답변이라, "live"와 "not just a demo"에 힘을 주세요. 이 두 표현이 신뢰를 만듭니다.

### ● Version B — Standard (약 75초, 자연스러운 영어) ← 기본값

So, this was an AI travel chatbot for the Busan Tourism Organization, and it's **live in production** on their official site right now. The idea is simple: you describe the trip you want in your own words, and it recommends attractions and builds the whole itinerary for you. Before this, their travel search was just **fixed filters** — pick a city, pick two days, pick "restaurants." So if someone typed something natural like "a quiet place I can go with my kids," the old system had nothing to match it against.

So the **key decision** was to make the search **meaning-based** instead of keyword-based. I took a two-stage recommendation approach — first a **metadata filter** on things like region, number of nights, and category, and then a **vector similarity** search on top. The part I'm proud of is what went into that vector: I pulled positive and negative keywords and **hashtags** out of the user reviews and folded them into the embedding. So the system matches on what a place actually feels like, not just its name or its official tags.

The hardest part wasn't really the model — it was working with **non-technical stakeholders**. I talked directly with the tourism organization's data team to line up the data schema and the update cycle,

and I had to negotiate scope with the planner — between the itinerary logic they wanted and the recommendation quality we could realistically ship. We handled that by shipping a **demo every two weeks** and folding their feedback back in.

In the end, it went **live in production** on the official Visit Busan site, and it still recommends spots and auto-builds itineraries from plain-language requests. I was the **AI lead** on a team of five, so I owned the vector database, the recommendation algorithm, and the chatbot itself. The takeaway for me was that the review-text trick — turning messy opinions into something searchable — was what actually made the recommendations feel relevant.

### 핵심 표현

- "live in production" = 상용 운영 중 (업계 표준 표현, 꼭 쓰세요)
- "fixed filters" = 고정 필터
- "meaning-based instead of keyword-based" = 의미 기반 vs 키워드 기반
- "fold A into B" = A를 B에 녹여 넣다
- "non-technical stakeholders" = 비기술 이해관계자
- "ship a demo every two weeks" = 2주마다 데모를 내놓다

**전달 팁:** Problem → Decision → Hard part 사이에서 0.5초씩 끊으면 구조가 또렷이 들립니다. "live in production"은 천천히, 또박또박.

## ● Version C — Deep Dive (약 2~3분, 고급 영어) ← "더 자세히" 요청 시

Sure, let me go a bit deeper, especially on the recommendation design.

The starting point was an algorithm I'd actually built before. On an earlier project at the same company — a **movie** recommendation system — I'd designed a **two-stage** approach: a **metadata filter** to narrow the candidate set, then a **vector similarity** search to rank by meaning. For the travel chatbot, I adapted that same pattern to a completely different domain. The metadata stage filters on **region**, number of **nights**, and **category**, which cheaply cuts the space down to plausible places before any heavy similarity work runs.

The interesting part is the vector itself. A travel spot isn't just its name and address — what people actually care about lives in the **reviews**. So instead of embedding only the official description, I **extracted positive and negative keywords and hashtags from the user reviews** and integrated them into the vector input. That's the one detail I'd highlight: a place that reviewers keep calling "quiet" or "good with kids" ends up **close in vector space** to a query that asks for exactly that — even if those words never appear in the official write-up. That's what turned it from keyword matching into genuine **semantic search**. And once a set of spots is selected, the system doesn't stop at a list — it **auto-composes them into a day-by-day itinerary**.

The other half of the project was getting it into production with non-technical partners. I coordinated directly with the tourism organization's **data team** on the schema and the refresh cadence, and I negotiated the **MVP scope** with the planner — balancing the itinerary logic they wanted against the recommendation quality we could ship on time. We ran on a **two-week demo cycle** so feedback never piled up. For deployment, I containerized the **API server and the vector database** with **Docker Compose**, split the per-environment settings into **.env** files, and automated the deploy. The result is a real, **live product** on Busan's official tourism site — and as the AI lead on a five-person team, I owned that whole path, from the vector DB to the chatbot.

### 핵심 표현 (고급)

- "adapted that same pattern to a different domain" = 같은 패턴을 다른 도메인에 적용
- "close in vector space" = 벡터 공간에서 가깝다
- "semantic search" = 의미 기반 검색 (필수 용어)
- "auto-compose them into an itinerary" = 그것들을 일정으로 자동 구성
- "refresh cadence" = 갱신 주기
- "negotiate the MVP scope" = MVP 범위를 협의하다
- "per-environment settings" = 환경별 설정

**전달 팁:** 길어서 중간에 "Want me to go deeper on the recommendation part?"로 호흡을 한 번 끊으면 좋습니다. itinerary 발음(eye-TIN-uh-rer-ee)만 미리 입에 붙여두세요.

## 3. 꼬리질문 대비 (Follow-up Q&A)

### Q1. Why two stages — why not just one vector search over everything?

Mostly efficiency and control. The metadata filter — region, nights, category — cheaply narrows the candidates to plausible places first, so the vector similarity step only runs on a small, relevant set. It also gives me a hard guarantee on the non-negotiable conditions; if someone says Busan for two nights, I don't want a semantically "similar" place in another city sneaking in. Meaning-based ranking is great, but some conditions should just be filters.

\*(한국어 메모: "효율 + 제약 보장" 두 축. 벡터가 만능이 아니라 필터로 보장할 건 보장한다는 게 시니어 신호.)\*

### Q2. How did the review keywords actually improve the recommendations?

The official descriptions are written by the venue, so they're polished and all sound the same — "beautiful view, great experience." The real signal is in the reviews: whether a place is actually quiet, crowded, kid-friendly, hard to park at. By extracting positive and negative keywords and hashtags from those reviews and putting them into the vector, the system can match on how a place really feels, not just how it's marketed. That's the difference between matching the word and matching the meaning.

\*(한국어 메모: "공식 설명은 다 비슷하다 → 리뷰에 진짜 신호가 있다"는 통찰을 강조. 이 프로젝트의 핵심 기술 포인트.)\*

### Q3. This sounds like your movie recommender. What did you have to change for travel?

The skeleton carried over — metadata filter, then vector similarity — but the domain was different in two ways. First, the metadata: a movie filters on things like genre, while travel filters on region, length of stay, and category. Second, and bigger, travel has to produce a **plan**, not just a ranked list. A good movie rec is one title; a good travel rec is several places arranged into a sensible day-by-day itinerary. So I added the itinerary-composition step on top of the ranking.

\*(한국어 메모: 재사용한 것(2단계 뼈대) vs 새로 만든 것(일정 구성)을 명확히 나눠 답하면 좋습니다.)\*

### Q4. You mention non-technical stakeholders a lot. What was hard about that?

The hard part is that the planner and I optimize for different things. They want rich itinerary logic; I care about whether the recommendation quality holds up. Early on, those pull in different directions. What worked was making it concrete — instead of arguing in the abstract, we shipped a demo every two weeks, looked at real outputs together, and let that drive the scope. Once there's something to point at, "I want X" turns into "this specific result is off," which is much easier to act on.

\*(한국어 메모: 비기술 이해관계자와의 긴장을 "데모로 구체화해서 해소"한 점이 핵심. 추상 논쟁 → 구체

산출물.)\*

#### Q5. How did you deploy it, and how do you keep environments straight?

I containerized both the API server and the vector database with Docker Compose, so the whole stack comes up the same way everywhere. Environment-specific things — endpoints, credentials, and so on — live in **.env** files, so the same image runs in dev or in production just by swapping the env file. And the deploy itself is automated, so pushing a new version isn't a manual, error-prone ritual.

\*(한국어 메모: Docker Compose + .env 분리 + 배포 자동화, 이 3점만 또렷이.)\*

#### Q6. There's no headline metric here. How do you know it worked?

Honestly, I don't have a single headline number to quote here. But the strongest signal is that it's **live in production** on a public institution's official site and still running — a tourism organization doesn't put a chatbot on its front-facing site unless it clears their bar. The real validation was the bi-weekly demos with the client and the fact that it passed their review and shipped. If I did it again, I'd instrument click-through and itinerary completion from day one, so I'd have that number to point to.

\*(한국어 메모: 지표 없음을 솔직히 인정 + "상용 배포 자체가 검증" + "다음엔 지표 심겠다"가 성숙한 답. 절대 숫자를 지어내지 마세요.)\*

## 4. 핵심 표현·어휘 (Key phrases & vocabulary)

| English                         | 뜻 / 쓰임                  |
|---------------------------------|-------------------------|
| live in production              | 상용 운영 중 (이 프로젝트의 헤드라인)  |
| natural language                | 자연어 (사람이 평소 쓰는 말)       |
| fixed filters                   | 고정 필터 (도시·일수 같은 정해진 조건) |
| meaning-based / semantic search | 의미 기반 검색                |
| keyword matching                | 키워드 매칭 (의미 검색의 반대편)     |
| two-stage approach              | 2단계 방식                  |
| metadata filter                 | 메타데이터 필터 (1단계)          |
| vector similarity               | 벡터 유사도 (2단계)            |
| embedding                       | 임베딩 (텍스트의 벡터 표현)        |
| fold A into B                   | A를 B에 녹여 넣다             |
| close in vector space           | 벡터 공간에서 가깝다             |
| auto-compose an itinerary       | 일정을 자동 구성하다             |
| non-technical stakeholders      | 비기술 이해관계자               |
| MVP scope                       | 최소 기능 범위                |
| refresh cadence / update cycle  | 갱신 주기                   |
| containerize                    | 컨테이너화하다                 |
| AI lead / own the whole path    | AI 파트 리드 / 전 과정을 책임지다   |

## 5. 발음 주의 (Delivery watch-outs)

| 단어             | 강세 / 주의                                     |
|----------------|---------------------------------------------|
| itinerary      | eye-TIN-uh-rer-ee — 강세 2번째 음절. 가장 자주 틀리는 단어 |
| vector         | VEK-ter — V는 윗니로 아랫입술 (B로 새지 않기)            |
| embedding      | em-BED-ding                                 |
| algorithm      | AL-guh-ri-thum                              |
| semantic       | si-MAN-tik                                  |
| metadata       | MET-uh-day-tuh                              |
| hashtag        | HASH-tag — 끝 -g 까지                          |
| stakeholder    | STAYK-hohl-der                              |
| schema         | SKEE-muh                                    |
| production     | pruh-DUK-shun                               |
| recommendation | rek-uh-men-DAY-shun                         |
| Busan          | "BOO-san" — 또박또박                            |

**마지막 점검:** 이 프로젝트엔 자랑할 숫자가 없습니다. 대신 **"live in production"**과 **"official Busan tourism site"**를 또렷이, 자신 있게 말하세요. 그리고 itinerary(eye-TIN-uh-rer-ee) 발음만 미리 입에 붙여두면 끝입니다.

### 2.2.2 Skylife 영화 추천 시스템

**한 줄:** Skylife 위성방송 가입자에게 선호 장르·배우로 유사 영화를 추천하는 API. pgvector 2단계 추천(메타 필터 → 벡터 유사도)으로 추천 다양성을 끌어올림.

**헤드라인:** pgvector 2단계 추천 · 추천 다양성(diversity) 개선 · 이후 벡터 검색 프로젝트들의 토대(석사 연구 → 실서비스 첫 구현).

소속 AIO2O · 2023.06~2023.12 · 4명 팀 · AI 파트(벡터 DB 설계·추천 알고리즘 개발) 전담.

## 0. 이 프로젝트로 증명할 것 (What this proves about you)

- **"연구를 실서비스로 옮기는 사람."** 석사 논문(Convolutional Autoencoder 기반 추천)의 아이디어를 처음으로 실제 가입자용 API로 구현한 출발점. ← 이 프로젝트의 가장 강한 서사.
- **문제를 다른 패러다임으로 환원하는 사고:** rule-based의 "늘 비슷한 결과" 한계를 벡터 유사도라는 전혀 다른 접근으로 해결.
- **토대를 만드는 사람:** 여기서 만든 2단계 추천(필터 → 벡터 유사도)이 이후 Visit Busan 챗봇 등 모든 벡터 검색 프로젝트의 뼈대가 됨.

이 프로젝트는 **숫자가 아니라 '서사'로 미는 카드**입니다. "Where did your vector / RAG work begin?" 또는 "어떻게 추천 시스템에 발을 들였나"에 이걸 꺼내세요. 무리해서 지표를 만들지 말고, **출발점 → 확장**

의 이야기로 끌고 가세요.

## 1. 생각 구조 (5-Beat skeleton, 이 프로젝트 버전)

| Beat        | 이 프로젝트에서 말할 내용                                                                       |
|-------------|--------------------------------------------------------------------------------------|
| 1 Headline  | Skylife 가입자용 영화 추천 API를 만들었다 — pgvector로 추천을 2단계로 재설계                                |
| 2 Problem   | 기존엔 메타 필터·rule-based 로직만 사용 → 같은 조건이면 늘 비슷한 결과가 반복됨. 다양성이 없었다                        |
| 3 Decision  | 영화 메타(장르·줄거리·감독·배우)를 임베딩해 pgvector에 저장. 추천을 2단계로 분리 — ① 메타 필터로 후보 축소 ② 설명 벡터 유사도로 순위 |
| 4 Hard Part | rule-based의 경직성을 벡터로 어떻게 깨나 + 필터(제약)와 유사도(다양성)를 어떻게 한 파이프라인에 결합하나                    |
| 5 Result    | 같은 조건에서도 유사하되 새로운 영화가 추천됨 → 다양성 개선. 이 2단계 알고리즘이 이후 Visit Busan 챗봇으로 확장됨              |

핵심 메시지 한 줄: "This is where my master's research on embedding-based recommendation became a real product — and the two-stage design became the foundation for everything I built with vectors afterward."

## 2. 모범답안 (Model Answers)

### ● Version A — Elevator (약 30초, 쉬운 영어)

In short, I built a movie recommendation API for SkyLife, a satellite TV service. A subscriber tells us their favorite genres and actors, and we recommend similar movies. The old system only used simple filters and fixed rules, so it kept suggesting the same kinds of movies. I changed that. I turned each movie's information — its genre, plot, director, and cast — into a vector and stored it in Postgres with **pgvector**. Then I made the recommendation work in **two steps**: first, narrow down the candidates with a filter, then rank them by how similar the movie descriptions are. The result was more variety — for the same request, people got movies that were similar but still fresh. And that two-step design ended up being the base for a lot of my later work.

#### 핵심 표현

- "the same kinds of movies" = 비슷비슷한 영화들 (반복 문제를 쉽게 표현)
- "turn A into a vector" = A를 벡터로 바꾸다 (= embed)
- "in two steps: first ..., then ..." = 2단계로: 먼저 ~, 그다음 ~
- "similar but still fresh" = 비슷하지만 그래도 새로운 (다양성을 쉬운 말로)

전달 팁: "two steps", "pgvector", "fresh"에서 또박또박. 숫자가 없는 프로젝트라 '바뀐 점'(반복 → 다양성)을 또렷이 대비시키는 게 힘입니다.

## ● Version B — Standard (약 75초, 자연스러운 영어) ← 기본값

So this was a movie recommendation system for Skylife, a satellite broadcaster. I was on the **AI side** — on a team of four, I designed the vector database and built the recommendation logic.

Before this, the recommendations ran purely on **metadata filters and rule-based logic**. The problem was that under the same conditions, it kept returning the same kind of results — the variety just wasn't there. So we were asked to improve the recommendation quality.

The **key decision** was to bring in **vector similarity**. I took each movie's metadata — genre, plot summary, director, and cast — embedded it, and stored it in **PostgreSQL using pgvector**. Then I split the recommendation into **two stages**: first, a metadata filter to narrow down the candidate pool, and then vector similarity on the movie descriptions to actually rank them.

That two-stage design was the important part. The filter keeps the results **relevant**, and the similarity step brings in **diversity** — so even for the same request, you get movies that are close but still new. We also **containerized** the API with Docker, so dev, staging, and production stayed identical.

The part I like most is where this came from. My master's research was on an embedding-based recommender, and this was the first time I put that idea into a real service. The same two-stage approach later grew into our **Visit Busan** travel chatbot.

### 핵심 표현

- "ran purely on ... rule-based logic" = 오로지 규칙 기반으로 돌아갔다
- "bring in vector similarity" = 벡터 유사도를 도입하다
- "narrow down the candidate pool" = 후보군을 좁히다 (1단계)
- "keeps it relevant ... brings in diversity" = 관련성은 지키고 다양성은 더한다 (2단계의 핵심 대비)
- "put that idea into a real service" = 그 아이디어를 실서비스로 옮기다

**전달 팁:** Problem → Decision → Result 사이에서 0.5초씩 끊으세요. 마지막 문장(석사 연구 → Visit Busan)이 이 답변의 **클로징 펀치**이니 서두르지 말 것.

## ● Version C — Deep Dive (약 2~3분, 고급 영어) ← "더 자세히" 요청 시

Sure, let me go a bit deeper on the design.

The starting point was actually a limitation I'd thought about during my master's. My thesis was a recommender built on a **convolutional autoencoder** — the idea was to compress a user's preferences into a **latent vector** and then recommend by vector similarity. Skylife was the first time I got to take that idea out of research and put it into a production service.

The problem on the ground was concrete. The existing API recommended movies with **metadata filters and rule-based logic only**. That's fine for relevance, but it's rigid — for the same genre-and-actor input, you'd get more or less the same list every time. There was no notion of "close, but different."

So the core of my design was a **two-stage pipeline**. In the first stage, I use the metadata filter as a **hard constraint** — genre, cast, and so on — to cut the catalog down to a sensible candidate set. In the second stage, I rank those candidates by **vector similarity**. To do that, I embedded the semantic side of each movie — genre, plot summary, director, cast — and stored those embeddings in **PostgreSQL with the pgvector extension**, so the similarity search runs right in the database, next to the data.

The reason it's two stages and not one is the real insight. If you only filter, you get relevance but no diversity — same rules, same results. If you only do vector similarity, you get diversity but you can **drift** past the user's hard requirements. Splitting it lets the filter guarantee the constraints while the similarity step introduces variety — movies that are semantically close to what they like, but not the identical list they've already seen. That's how I improved the recommendation **diversity** without giving up relevance.

On the delivery side, I **containerized** the API server with Docker. The Dockerfile pinned the environment, so development, staging, and production were identical — no "works on my machine" surprises.

And the part I find satisfying is the through-line. That two-stage shape — a cheap filter to narrow, then vector similarity to rank — turned out to be reusable. It became the backbone of our **Visit Busan** travel chatbot later on, and really of every vector-search project I touched after this. Skylife was where it started.

#### 핵심 표현 (고급)

- "compress preferences into a latent vector" = 선호를 잠재 벡터로 압축
- "take it out of research into production" = 연구에서 실서비스로 옮기다
- "hard constraint" = 하드 제약 (반드시 지켜야 하는 조건)
- "drift past the user's hard requirements" = 사용자의 필수 조건을 벗어나 표류하다
- "runs right in the database, next to the data" = DB 안에서, 데이터 바로 옆에서 돈다
- "the through-line / the backbone" = 관통하는 줄기 / 뼈대

**전달 팁:** 이 버전은 길어서 "the reason it's two stages and not one is the real insight"가 **하이라이트**입니다. 거기서 한 박자 쉬고 또박또박. 끝에 "Want me to go further on any part?"로 점검하면 자연스럽습니다.

### 3. 꼬리질문 대비 (Follow-up Q&A)

#### Q1. Why two stages? Why not just do vector similarity over the whole catalog?

Because each stage solves a different problem. Pure vector similarity can **drift** — it might rank a movie that's semantically close but breaks a hard requirement, like the wrong genre. Pure filtering is safe but gives you no variety. So I let the filter handle the hard constraints first, then let similarity bring diversity **within** that safe set. You get relevance and freshness at the same time.

\*(한국어 메모: "각 단계가 다른 문제를 푼다 — 필터 = 제약, 벡터 = 다양성"으로 답하면 설계 의도가 분명해집니다.)\*

#### Q2. You say diversity improved — how did you measure it?

Honestly, I didn't have a formal offline diversity metric on this project — it was an early build. The signal was **qualitative**: for the same input, the old system returned essentially the same list, and the new one returned movies that were similar in feel but not the same titles. If I built it again, I'd track something like intra-list diversity or coverage so the claim is quantitative. But the direction of the change was clear.

\*(한국어 메모: 숫자가 없으니 솔직하게. "정성적이었다 + 다시 하면 이렇게 측정하겠다"가 가장 성숙한 답입니다. 절대 지표를 지어내지 마세요.)\*

#### Q3. Why pgvector instead of a dedicated vector database?

I keep the movie metadata and the embeddings together in **PostgreSQL**, so pgvector lets the filter and the similarity search happen in **one place** — no separate vector service to run or keep in sync. For the scale of a movie catalog, that was the simpler, more maintainable choice. A dedicated vector DB makes more sense once you outgrow that.

\*(한국어 메모: "데이터가 한곳에 있다 + 동기화할 별도 서비스 불필요 + 규모에 맞는 단순함" — 트레이드오프로 답하세요.)\*

#### Q4. What exactly did you embed — the whole movie?

The **semantic** parts — genre, plot summary, director, and cast. Those carry the "what this movie is actually about" signal, which is what you want similarity to capture. The harder categorical conditions stayed in the filter stage. So the two stages also split the data cleanly: structured fields for filtering, descriptive content for the embedding.

\*(한국어 메모: 임베딩 대상 = 장르·줄거리·감독·배우. "의미 신호는 벡터로, 범주 제약은 필터로" 분리한다고 답.)\*

#### Q5. Where did this project lead? How does it connect to your other work?

This was the **foundation**. The two-stage pattern — narrow with a cheap filter, then rank by vector similarity — became a template I reused. It grew into our **Visit Busan** travel chatbot, and it shaped how I approached every vector-search and retrieval problem after that. It also closed a loop for me personally, because it started as my master's research on embedding-based recommendation.

\*(한국어 메모: "출발점 → Visit Busan으로 확장 → 석사 연구와 연결"의 서사. 이 프로젝트의 핵심 카드이니 자신 있게.)\*

#### Q6. What would you do differently next time?

Two things. I'd add a real **diversity metric** up front, so I could prove the improvement instead of just describing it. And I'd experiment with how the two stages interact — for example, tuning how many candidates the filter passes to the ranking step, since that knob trades off relevance against variety. The core design held up well, though — that's exactly why it carried into later projects.

\*(한국어 메모: "다양성 지표 추가 + 필터 → 랭킹 후보 수 튜닝"으로 약점을 개선안으로 전환.)\*

## 4. 핵심 표현·어휘 (Key phrases & vocabulary)

| English                             | 뜻 / 쓰임                 |
|-------------------------------------|------------------------|
| recommendation system / recommender | 추천 시스템                 |
| two-stage pipeline                  | 2단계 파이프라인 (필터 → 랭킹)    |
| metadata filter                     | 메타데이터 필터 (1단계: 후보 축소)  |
| vector similarity                   | 벡터 유사도 (2단계: 순위 매김)    |
| embedding / embed                   | 임베딩 / 임베딩하다 (텍스트를 벡터로) |
| candidate pool / candidate set      | 후보군                    |
| hard constraint                     | 하드 제약 (반드시 지켜야 하는 조건)  |
| diversity                           | 다양성 (추천이 다채로움)         |

|                                         |                      |
|-----------------------------------------|----------------------|
| relevance                               | 관련성 (입력과 얼마나 맞나)     |
| rule-based                              | 규칙 기반                |
| rigid                                   | 경직된 (유연하지 않은)        |
| drift                                   | (제약을) 벗어나 표류하다       |
| containerize                            | 컨테이너화하다 (Docker로)    |
| right next to the data                  | 데이터 바로 옆에서 (DB 내부에서) |
| the backbone / the foundation           | 뼈대 / 토대              |
| take it out of research into production | 연구를 실서비스로 옮기다        |

## 5. 발음 주의 (Delivery watch-outs)

| 단어             | 강세 / 주의                                    |
|----------------|--------------------------------------------|
| recommendation | rek-uh-men-DAY-shun                        |
| pgvector       | "P-G-vector" (피지-벡터). v를 b로 새지 않기          |
| PostgreSQL     | "POST-gres" 로 줄여 말해도 됨                     |
| similarity     | sim-uh-LAIR-uh-tee                         |
| diversity      | dy-VER-suh-tee                             |
| genre          | "ZHAHN-ruh" (장르) — 첫소리는 j가 아니라 부드러운 '쥬' 소리 |
| vector         | VEK-ter. v를 b로 새지 않기                       |
| embedding      | em-BED-ding                                |
| metadata       | MET-uh-day-tuh                             |
| autoencoder    | AW-toh-en-KOH-der                          |
| catalog        | KAT-uh-log                                 |
| Docker         | DOK-er                                     |

**마지막 점검:** 이 프로젝트는 숫자가 아니라 '**서사**'로 미는 카드입니다. "My vector work started here" — 출발점에서 시작해 Visit Busan으로 확장됐다는 즐거움, 천천히 자신 있게.

### 2.2.3 MarketScope AI 상권분석 SaaS

**한 줄:** 지도에서 상권을 고르고 자연어 한 줄로 물으면 AI가 실시간으로 분석해 주는, 실제 운영 중인 상권분석 SaaS.

**헤드라인 숫자:** 프로덕션 라이브 v0.4.0 · 서울 1,650개 상권 / 약 14.6만 건 실데이터 · ReAct 7종 Tool · E2E 66/66 + 스모크 28/28.

1인 풀스택 사이드 프로젝트(기획·설계·프론트엔드·백엔드·AI Agent·인프라 전부) · 2026.01~현재 운영 중 · live: <http://marketscope.robilabs.co.kr/> · GitHub SJayKim.

## 0. 이 프로젝트로 증명할 것 (What this proves about you)

- 기획부터 인프라까지 혼자 만들어 실제 배포까지 한 **end-to-end 실행력**. ← 사이드 프로젝트에서 가장 강한 신호. "0에서 1까지 혼자 다 했다."
- **풀스택 폭**: AI 에이전트(ReAct 7 Tool) + 프론트(지도 연동) + 데이터 파이프라인(ETL 약 14.6만 건)을 한 프로젝트 안에서 전부.
- **환각을 줄이는 정확성 설계**: 그냥 LLM을 부르는 게 아니라, Entity Linking과 Abstention으로 "믿고 쓸 수 있게" 만든 점.
- **테스트로 품질을 보증**: E2E 66/66 + 프로덕션 스모크 28/28.

이 프로젝트는 "Tell me about a side project" 또는 "혼자서 뭔가를 끝까지 만들어 본 경험"에 꺼내는 카드입니다. Project 05가 "운영 규모"의 카드라면, 이걸 **"혼자, 0에서 1까지, 실제 배포까지"**의 카드입니다.

## 1. 생각 구조 (5-Beat skeleton, 이 프로젝트 버전)

| Beat        | 이 프로젝트에서 말할 내용                                                                 |
|-------------|--------------------------------------------------------------------------------|
| 1 Headline  | 지도에서 상권을 고르고 자연어로 물으면 AI가 분석해 주는 SaaS를 혼자 만들어 실제 운영 중                          |
| 2 Problem   | 기존 상권분석 서비스(Openup 등)는 정적 대시보드 + 수동 필터 → 사용자가 직접 지표를 찾아 비교해야 함. 소상공인에겐 너무 번거로움 |
| 3 Decision  | 핵심 결정은 "대화형"으로. LangGraph ReAct 에이전트가 7종 Tool을 스스로 골라 엮어 복합 질의를 실시간 분석         |
| 4 Hard Part | (1) 신뢰성 — Entity Linking + Abstention으로 환각 차단 (2) 지도↔챗봇 양방향 동기화. 게다가 이 전부를 혼자  |
| 5 Result    | 프로덕션 라이브 v0.4.0, 서울 1,650개 상권 / 약 14.6만 건, ReAct 7 Tool, E2E 66/66 + 스모크 28/28 |

핵심 메시지 한 줄: **"It's one person taking an AI product all the way — from the map UI down to the data pipeline and the infra — and actually shipping it to production."**

## 2. 모범답안 (Model Answers)

### ● Version A — Elevator (약 30초, 쉬운 영어)

In short, I built **MarketScope AI** — a live web service that analyzes commercial districts with AI. It's a side project, and I built the whole thing by myself. You pick an area on the map — say, Gangnam Station — and just ask in plain language, like "How would a cafe do here?" The AI reads the real data and answers in seconds, right on the map. Most tools out there just show you a static dashboard and make you dig through the numbers yourself. Mine does the thinking for you. And it's **live in production** right now, with real data for about **1,650** commercial districts in Seoul.

### 핵심 표현

- "I built the whole thing by myself" = 전부 혼자 만들었다 (solo 강조)
- "in plain language" = 평범한 말로 = 자연어로
- "does the thinking for you" = 대신 생각해 준다
- "live in production" = 실제 운영 중인

**전달 팁:** "by myself"와 "1,650"에서 또박또박 멈추세요. 사이드 프로젝트의 힘은 "혼자 끝까지"입니다.

## ● Version B — Standard (약 75초, 자연스러운 영어) ← 기본값

So, MarketScope AI is a **map-based commercial-district analysis** service — basically, an AI that helps small business owners and real estate investors size up a neighborhood before they commit. It's a **solo side project**. I did everything myself, from the planning and the UI down to the backend, the AI agent, and the infra. And it's live in production.

The problem I started with is that the existing tools — like the public ones in Korea — give you a static dashboard and a pile of manual filters. You have to know which metric to look at, find it, and compare it yourself. For a regular shop owner, that's just too much work.

So the **key decision** was to make it conversational. You select a district on the map and ask something like "How are cafe sales trending at Gangnam Station?" or "Compare it with Hongdae," and the AI analyzes it in real time. Under the hood, that's a **LangGraph ReAct agent** — it reasons, picks the right tool, looks at the result, and repeats. I gave it **seven tools** — foot traffic, sales, stores, district comparison, and so on — and it chains them automatically. So "should I open a cafe here?" becomes sales, then stores, then a recommendation, fused into one answer.

The **hardest part** was trust. An AI that confidently makes up numbers is worse than no AI. So I added two things: **entity linking**, which maps messy district names — abbreviations, typos — to the right area, and **abstention**, where if a tool fails or comes back empty, the AI says it doesn't have the data instead of guessing.

And it **shipped** — it's live at version 0.4.0, with real data for about **1,650** districts in Seoul, roughly **14 6,000** records. I also kept it tested end to end: **66 out of 66** E2E tests pass, plus 28 smoke tests against production. So the whole thing — agent, map, data pipeline — actually runs, by myself, in production.

### 핵심 표현

- "solo side project" = 1인 사이드 프로젝트
- "from the planning ... down to the infra" = 기획부터 인프라까지 (전 영역 오너십)
- "make it conversational" = 대화형으로 만들다
- "under the hood" = 내부적으로는 (구현 관점 진입 표현)
- "chains them automatically" = (도구를) 자동으로 엮는다
- "confidently makes up numbers" = 자신 있게 숫자를 지어낸다 (= 환각)
- "it shipped" = 출시했다 / 배포를 끝냈다

**전달 팁:** Problem-Decision-Hard part-Result 네 군데에서 잠깐씩 멈추세요. 마지막 "by myself, in production"은 자신 있게.

## ● Version C — Deep Dive (약 2~3분, 고급 영어) ← "더 자세히" 요청 시

Sure, let me go deeper — and since I built all of it, I can go top to bottom.

The brief I set myself was simple: one person, end to end, and it has to actually ship and be trustworthy enough that a shop owner would act on it.

The core is the AI chat. It's a **LangGraph ReAct agent** — reason, act, observe, looping up to **five** times — running on the **Claude API with tool use**, prompted to behave like a Korean commercial-district consultant. I gave it **seven tools**: foot traffic, sales, stores, district comparison, industry recommendation, risk, and population. The agent picks and chains them on its own. So "how about a cafe at Gangnam Station?" turns into a sales call, then a stores call, then a recommendation, fused into one narrative answer.

The detail I'm most proud of is the **accuracy layer**, because that's where most of these things fall apart. The single biggest failure was users typing district names that don't match the database — abbreviations, nicknames, typos. So I built **entity linking**: prefix and trigram matching against the canonical names, backed by a **learning alias table** that remembers past resolutions, so the system absorbs messy input over time. And I paired it with **abstention** — if a tool fails or returns nothing, the agent explicitly says it doesn't have that data instead of inventing a number. For an analysis product, a confident wrong answer is the worst thing you can ship, so I designed for "I don't know" on purpose.

On the front end, it's Next.js with **Kakao Map** for Korean map accuracy and **deck.gl** for fast heatmap and polygon rendering. The map and the chat are wired both ways through a **Zustand** store: clicking a polygon kicks off analysis, and the AI's response carries **map\_cmd** events that move, zoom, and highlight the map back. The answers render as rich cards — summary, compare, recommend, risk — with inline Recharts charts, and the whole thing streams over **FastAPI server-sent events**, so you watch the agent think — thinking, tool, text, card, done — in real time.

Underneath, there's a data pipeline: an **ETL** that pulls four Seoul open-data services plus the shapefile polygons every quarter — about **146,000** records across **1,650** districts — into Postgres with PostGIS. I used a **repository pattern**, so a single environment variable, **USE MOCK**, flips the whole app between a database-less mock mode and real data — which is also what keeps the tests fast. And I wired up **Langfuse** for tracing, with user feedback fed back into the trace scores so I can improve on real usage.

And it's all verified — Playwright scenarios from preflight to negative cases, **66 out of 66** E2E plus **28** production smoke tests. It's live right now at version 0.4.0.

### 핵심 표현 (고급)

- "top to bottom / end to end" = 처음부터 끝까지, 전 영역
- "reason, act, observe, looping up to five times" = ReAct 루프를 영어로 푸는 법
- "picks and chains them on its own" = 스스로 고르고 엮는다
- "that's where most of these things fall apart" = 대개 거기서 무너진다
- "a learning alias table that remembers past resolutions" = 과거 해석을 기억하는 학습형 별칭 테이블
- "I designed for 'I don't know' on purpose" = 일부러 '모른다'를 설계에 넣었다
- "wired both ways" = 양방향으로 연결했다
- "server-sent events" = 서버가 밀어주는 실시간 스트리밍(SSE)
- "flips the whole app ... with one environment variable" = 변수 하나로 모드 전환

**전달 팁:** 길어서, 중간에 "Want me to zoom in on the agent side or the data side?"로 면접관 반응을 확인하세요. 한 번에 다 쏘지 마세요.

### 3. 꼬리질문 대비 (Follow-up Q&A)

#### Q1. Why build this as a side project, and what did you learn doing it solo?

Honestly, I wanted to prove I could take something from an idea all the way to production by myself — not just the model, but the UI, the data pipeline, and the infra. Doing it solo forced real tradeoffs about scope: I shipped a mock mode first so I could test the whole flow before the data was ready, then swapped in real data. The biggest lesson was that the hard part of an AI product isn't the model call — it's everything around it: clean data, accurate entity resolution, and knowing when to say "I don't know."

\*(한국어 메모: "혼자 = 전 영역 오너십"으로 답하고, 배운 점은 "모델보다 그 주변이 어렵다"로 마무리. 성숙한 시니어 신호입니다.)\*

#### Q2. What's the business model?

It's **freemium**. The free tier gives you five analyses a day and a basic report — enough to try it and get value. The premium tier removes the limit and unlocks the deeper stuff: heatmaps, sales simulation, and PDF reports. The idea is that a curious shop owner starts free, and someone seriously scouting a location pays for the depth. The tier gating and the payments are the next phase I'm building.

\*(한국어 메모: 무료/프리미엄 경계를 구체적 기능으로. "진지한 사용자가 깊이에 돈을 낸다"는 가치 논리. Phase 2(결제·Tier 게이팅)가 진행 예정임을 덧붙이면 솔직하고 좋습니다.)\*

#### Q3. How is this different from existing tools like Openup?

The existing ones are static dashboards with manual filters — you have to know the metric, find it, and compare it yourself. Mine is conversational: you ask in one plain sentence, and the AI picks the data, runs the comparison, and visualizes it on the map for you. So it turns "go research this" into "just ask."

\*(한국어 메모: "정적 대시보드 + 수동 필터" vs "자연어 한 줄"을 대비. 차별점은 한 문장으로 압축하는 게 강합니다.)\*

#### Q4. How do you stop the AI from hallucinating numbers?

Two layers. **Entity linking** makes sure a messy district name maps to the right real area before any analysis runs — prefix and trigram matching plus a learning alias table. And **abstention**: if a tool fails or returns nothing, the agent says it doesn't have the data instead of guessing. For an analysis tool, a confident wrong number is the worst thing you can ship, so I designed for "I don't know" on purpose.

\*(한국어 메모: Entity Linking + Abstention 두 축. "오답보다 기권이 낫다"는 설계 철학을 분명히 말하세요.)\*

#### Q5. Why a ReAct agent with seven tools instead of one big prompt?

Because real questions are compound. "Should I open a cafe here?" isn't one lookup — it's sales, then stores, then a recommendation. A ReAct agent reasons about what it needs, calls the right tool, looks at the result, and decides the next step, up to five loops. One giant prompt can't fetch live data or pick its own path like that. The tools keep every answer grounded in actual records.

\*(한국어 메모: "복합 질의"가 핵심 이유. ReAct = 필요한 도구를 스스로 골라 엮음. 단일 프롬프트와 대비하면 설계 의도가 또렷합니다.)\*

#### Q6. How did you make sure it actually works in production?

I leaned on tests hard, since I was solo and couldn't eyeball everything. I have Playwright scenarios in rings — preflight, features, journeys, and negative cases — 66 out of 66 passing, plus 28 smoke tests

that run against the live site. And because I can flip the app into mock mode with one environment variable, I can regression-test the whole flow without a database. That's how I keep shipping changes without breaking the live version.

\*(한국어 메모: "혼자라서 테스트에 의존"이 솔직하고 좋습니다. Ring 0-3, 66/66 + 스모크 28/28, USE MOCK 회귀까지 묶어 품질 보증 신호로.)\*

#### 4. 핵심 표현·어휘 (Key phrases & vocabulary)

| English                                | 뜻 / 쓰임                 |
|----------------------------------------|------------------------|
| commercial-district analysis           | 상권 분석 (이 프로젝트의 도메인)    |
| end to end / top to bottom             | 처음부터 끝까지, 전 영역         |
| solo side project                      | 1인 사이드 프로젝트            |
| ship / it shipped / live in production | 출시하다 / 배포 완료 / 실제 운영 중 |
| make it conversational                 | 대화형으로 만들다              |
| ReAct agent                            | 추론-행동-관찰 루프 에이전트       |
| tool use / chain tools                 | 도구 호출 / 도구를 엮다         |
| entity linking                         | 지명 같은 엔티티를 정규화-매칭      |
| abstention                             | 모르면 기권 (추측하지 않음)       |
| hallucinate / make up numbers          | 환각하다 / 숫자를 지어내다        |
| under the hood                         | 내부적으로는 (구현 관점)         |
| wired both ways / bidirectional sync   | 양방향으로 연결 / 양방향 동기화     |
| server-sent events (SSE)               | 서버가 밀어주는 실시간 스트리밍      |
| data pipeline / ETL                    | 데이터 수집·적재 파이프라인        |
| freemium / tier gating                 | 무료+유료 모델 / 등급별 기능 제한   |
| smoke test                             | 핵심 기능만 빠르게 점검하는 테스트    |

#### 5. 발음 주의 (Delivery watch-outs)

| 단어             | 강세 / 주의               |
|----------------|-----------------------|
| commercial     | kuh-MUR-shul          |
| district       | DIS-trikt (끝 -ct 끝까지) |
| conversational | kon-ver-SAY-shuh-nul  |
| entity         | EN-ti-tee             |
| trigram        | TRY-gram              |

|             |                                              |
|-------------|----------------------------------------------|
| abstention  | ab-STEN-shun                                 |
| hallucinate | huh-LOO-si-nayt                              |
| polygon     | PAH-lee-gon                                  |
| freemium    | FREE-mee-um                                  |
| Kakao       | kuh-KOW (한 단어로 빠르게)                          |
| 1,650       | "one thousand six hundred fifty" (개수니까 또박또박) |
| 146,000     | "a hundred forty-six thousand"               |
| v0.4.0      | "version zero point four point zero"         |

**마지막 점검:** 이 프로젝트의 힘은 **"혼자, 끝까지, 실제 배포"**입니다. "by myself", "end to end", "it shipped", "live in production" 네 표현을 자신 있게 던지세요.

## 3 공통 질문 대비

강점·약점·실패·갈등·지원 동기 같은 공통 질문에 STAR 구조로 자기 경험에 근거해 답할 수 있습니다.

이 Phase는 다음 섹션으로 구성됩니다.

### 3.1 행동·공통 질문

#### 3.1.1 강점·약점·실패·갈등·지원동기 (STAR)

### 3.1 행동·공통 질문

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

**3.1.1 강점·약점·실패·갈등·지원동기 (STAR)** — STAR(Situation-Task-Action-Result) 구조로 강점·약점·실패·갈등·지원 동기 질문에 자기 경험으로 답할 수 있다.

#### 3.1.1 강점·약점·실패·갈등·지원동기 (STAR)

프로젝트 외의 "사람" 질문들. 답의 재료는 모두 당신의 실제 경험에서 가져왔습니다.

여기서는 **STAR** 구조를 씁니다 (프로젝트 설명의 5-Beat와 다름).

## 0. STAR 구조 (행동 질문 전용)

"~한 적 있나요?(Tell me about a time ...)"류 질문은 이 4단계로:

| STAR      | 한국어         | 시작 표현                                          |
|-----------|-------------|------------------------------------------------|
| Situation | 상황 (배경)     | "At my company, we had a situation w here ..." |
| Task      | 내 과제        | "My job was to ..."                            |
| Action    | 내가 한 행동     | "So what I did was ..."                        |
| Result    | 결과 (숫자면 최고) | "As a result, ..."                             |

핵심: "우리"가 아니라 "내가" 한 일을 말하세요. 면접관은 당신의 기여를 봅니다.

## 1. What is your greatest strength? (강점)

**생각 메모:** 강점 = "파이프라인 전체를 본다" + 증거 프로젝트 하나.

I'd say my biggest strength is seeing a problem as a **whole pipeline**, not just a model. A lot of engineers optimize the model in isolation, but real service quality comes from preprocessing, retrieval, serving, and evaluation working together. In one RAG project, instead of only tuning the search, I

cleaned up the data first — I removed about **12 percent** noisy chunks out of 150,000 — and that alone lifted retrieval quality. That habit of zooming out to the whole system is what I bring.

**핵심 표현:** "seeing a problem as a whole pipeline" / "in isolation"(고립적으로) / "zoom out to the whole system"(전체로 시야를 넓히다)

---

## 2. What is your greatest weakness? (약점)

**생각 메모:** 진짜 약점 + 개선 노력. **치명적이지 않고 실제적인 것**. 당신의 진짜 약점은 "넓이는 충분한데 클라우드 매니지드 환경(AWS/K8s) 깊이가 상대적으로 얕다" — 그리고 지금 학습 중. 솔직하고 안전합니다.

Honestly, my breadth has sometimes run ahead of my depth in one specific area: **managed cloud**. I've done a lot with Docker and Docker Compose and built multi-container GPU serving myself, but most of that was self-hosted, not on a managed platform like AWS or Kubernetes. I noticed that gap, so I'm actively closing it — I've been doing hands-on work with AWS Bedrock and studying managed LLM services, so I can carry the same container setups into a cloud environment. It's a real gap, but it's one I'm already working on.

**핵심 표현:** "my breadth ran ahead of my depth"(넓이가 깊이를 앞섰다) / "I noticed that gap, so I'm actively closing it"(그 갭을 인지하고 메우는 중) / "hands-on"(직접 해보는)

**전달 팁:** 약점은 담담하게 인정 → 곧바로 "그래서 이렇게 하고 있다"로. 변명하지 마세요.

---

## 3. Tell me about a difficult technical problem you solved. (어려웠던 문제 — STAR)

**생각 메모:** 평가 파이프라인(비용)이나 스미싱(단축 URL)을 STAR로. 아래는 평가 파이프라인.

**(Situation)** We had a summarization model already running in production, and we needed to evaluate about **600,000** of its outputs in a short window. **(Task)** My job was to design the whole evaluation pipeline — and the hard constraint was cost: calling an LLM judge for every quality dimension on 600,000 items would have been impossibly expensive. **(Action)** So I split the metrics into two groups — deterministic ones I compute without any LLM, and a single quality score that needs just one LLM call. For that one call, I bundled all five quality criteria into a single judge prompt instead of five separate ones. **(Result)** That cut the inference cost to about a **fifth** while keeping quality, and the whole thing ran unattended on cron. So whenever the production model changes, we catch any regression on the next cycle.

**핵심 표현:** "the hard constraint was cost"(핵심 제약은 비용이었다) / "bundled five criteria into one"(다섯을 하나로 묶었다) / "catch a regression"(품질 저하를 잡다)

\*(다른 버전이 필요하면 07\_Project\_05 정답지를 그대로 쓰면 됩니다.)\*

---

## 4. Tell me about a time you failed or made a mistake. (실패·실수)

**생각 메모:** 진짜 실수 + 교훈. vLLM 프로젝트에서 요약·분류·태깅을 한 로직에 묶었다가 디버깅이 힘들어 분리한 경험 — 실제로고, 성장 서사가 깔끔.

Early in the vLLM serving project, I built summarization, classification, and tagging as **one big combined logic**. It worked at first, but as the rules grew, it became really hard to debug — when something broke, I couldn't tell which part caused it. That was my mistake: I optimized for "fewer moving parts" instead of "easy to reason about." So I **split it into separate agents** connected with LangGraph, and after that the summary-length convergence rate went up to about **92 percent** and debugging got much simpler. The lesson I took away was to **decompose complex work into small, observable units** from the start — and I've applied that to every project since, including the Reflexion agent.

**핵심 표현:** "one big combined logic"(하나로 뭉친 로직) / "hard to debug"(디버깅이 힘든) / "decompose into small units"(작은 단위로 분해) / "the lesson I took away"(내가 얻은 교훈)

**전달 팁:** 실패담은 솔직하게 인정 → 구체적 교훈 → 이후 적용의 3단. 작은 실수, 큰 교훈.

---

## 5. Tell me about a disagreement or conflict on a team. (갈등·협업)

**생각 메모:** Visit Busan — 기획자가 원하는 일정 로직 vs AI 추천 품질 사이 트레이드오프. MVP 범위 협의 + 2주 데모로 해결. 갈등을 "협업으로 푼" 이야기.

On the Busan travel chatbot, there was real tension between what the **planner** wanted and what was technically sound. The planner wanted very detailed, fixed itinerary logic, but that conflicted with keeping the AI recommendations flexible and high-quality. Instead of arguing in the abstract, I proposed we **agree on an MVP scope** and run a **demo every two weeks**. That way, the planner could see real output early and give feedback, and I could show the tradeoffs concretely instead of debating them on paper. We converged quickly, and the chatbot ended up **live in production** on the official site. What I learned is that most disagreements dissolve once people are looking at the same working demo.

**핵심 표현:** "real tension between A and B"(A와 B 사이 실질적 긴장) / "instead of arguing in the abstract"(추상적으로 다루는 대신) / "agree on an MVP scope"(MVP 범위에 합의) / "most disagreements dissolve once ..." (대부분의 갈등은 ~하면 사라진다)

---

## 6. Tell me about a time you took ownership / led something. (주도성·리더십)

**생각 메모:** MarketScope 1인 풀스택(기획~인프라~배포) 또는 Visit Busan AI 리드. 아래는 MarketScope.

The clearest example is **MarketScope**, an AI commercial-district analysis SaaS I built completely solo — planning, design, frontend, backend, the AI agent, and the infrastructure. Nobody assigned it to me; I saw that small-business owners had to dig through several statistics sites by hand, so I built a service where one natural-language question gives you an AI analysis on a map. I took it all the way to a **live production release**, with **66 out of 66** end-to-end tests passing and real data for **1,650** districts in Seoul. Doing every layer myself taught me where the real handoffs and failure points are across a full stack.

**핵심 표현:** "completely solo"(완전히 혼자) / "nobody assigned it to me"(아무도 시키지 않았다 — 주도성 신호) / "took it all the way to a live release"(실제 배포까지 끌고 갔다)

---

## 7. Why should we hire you? / Why are you a good fit? (왜 당신인가)

**생각 메모:** 차별점 한 줄로 못 박기. 평가 + 풀 파이프라인 + 배포까지.

Three things. First, I cover the **whole pipeline** — collection, search, serving, generation, and evaluation — so I can see where problems really come from. Second, I have a skill that's still rare: I build the systems that **verify LLM quality in production at scale**, not just the models themselves. And third, I actually **ship** — my side project is live, and the Busan chatbot I led is still in production. So I bring breadth, a rare evaluation skill, and a track record of finishing.

**핵심 표현:** "a skill that's still rare"(아직 드문 역량) / "I actually ship"(실제로 출시한다) / "a track record of finishing"(끝까지 해내는 이력)

---

## 8. Where do you see yourself in a few years? (몇 년 후)

I want to become the person a team trusts to take an AI system **from idea to reliable production** — especially on the evaluation and operations side, which is where I think the field is heading. Models are getting easier to build; knowing they're actually good, safely and cheaply, is the hard part. That's the depth I want to own.

**핵심 표현:** "from idea to reliable production"(아이디어에서 신뢰할 수 있는 운영까지) / "where the field is heading"(이 분야가 향하는 곳)

---

## 9. Why this company / this role? (지원 동기 — 템플릿)

회사마다 달라야 합니다. 아래 빈칸 템플릿에 면접 전 조사 내용을 채우세요.

I'm drawn to this role for two reasons. First, [**회사의 제품/문제 — 예: you're running LLMs in production at real scale**], and that's exactly the problem I've spent the last year on — evaluating and operating production LLMs. Second, [**팀/기술 방향 — 예: your focus on X**] lines up with where I want to grow. I'm not looking for just any AI role; I'm looking for one where [**내 강점**] actually matters, and this looks like that.

**조사 체크리스트 (면접 전):** ① 회사 제품/주력 도메인 ② 최근 출시·블로그·채용공고 키워드 ③ 그 키워드와 내 프로젝트의 접점 1개.

**핵심 표현:** "I'm drawn to this role because ..." (이 역할에 끌리는 이유는) / "lines up with where I want to grow" (내 성장 방향과 맞는다)

---

## 10. Do you have any questions for us? (역질문 — 절대 "없다" 금지)

좋은 역질문은 "나는 이 일을 진지하게 생각한다"는 신호입니다. 3개 준비:

| 질문 (English)                                                                        | 무엇을 보여주나              |
|-------------------------------------------------------------------------------------|-----------------------|
| "How do you currently evaluate whether your models are working well in production?" | 내 차별점(평가) 영역으로 대화를 끝다 |
| "What does the path from prototype to production look like on your team?"           | 배포·운영 지향임을 드러낸다       |
| "What would success in this role look like in the first six months?"                | 진지함 + 기대치 정렬          |

**전달 팁:** 면접 중 들은 내용에 즉석 질문을 더하면 최고입니다: "You mentioned \_\_\_ earlier — could you tell me more about how that works?"

## 11. Salary expectations (연봉 — 받았을 때만, 부드럽게)

I'm flexible and more focused on the fit and the work right now. I'd rather understand the full scope first — but I'm happy to share a range if that helps.

**핵심 표현:** "I'm flexible and more focused on the fit"(유연하며 적합성에 더 집중) — 숫자를 먼저 못 박지 않는 안전한 표현.

**마지막 점검:** 행동 질문은 전부 **당신의 실제 경험**에서 나옵니다. 지어내지 마세요.

강점=파이프라인 전체, 약점=클라우드 깊이(학습 중), 실패=vLLM 통합 로직, 갈등=Visit Busan MVP 협의, 주도성=MarketScope. 이 5개만 입에 붙이면 대부분 커버됩니다.